```
// G12AdFunnel.cs
using System;
using System.Collections.Generic;
using System.Threading.Tasks;
using Sy.Platform;
using Sy.Telemetry;
namespace Sy.G12.Boot
{
    /// &lt;summary&gt;
    /// evt_ad funnel for G12 (task S2-T004, P02 T015 eight-point / G06AdFunnel
    /// paradigm). The three-piece law every offer seat consumes: (1) CanOffer
    /// pre-checks the point's category budget on the SAME policy instance the
    /// suite's ads service consumes from - an over-cap offer never fires, no dead
    /// ad buttons; (2) the accept flow goes through the category-aware
    /// ShowRewarded(pointId) so the daily caps read exactly as the design table
    /// (rescue 4 / direct 1 / double 5, global 12); (3) every state transition
    /// lands as an evt_ad telemetry event (offer/accept/blocked/granted/ad_fail +
    /// point + reason) so the funnel is auditable end to end. S2-T006 adds the
    /// section-8 "category" field (rescue/direct/double) so the IAA health face
    /// reads the design-table category straight from the payload.
    /// &lt;/summary&gt;
    public sealed class G12AdFunnel
    {
        public const string StateOffer = "offer";
        public const string StateAccept = "accept";
        public const string StateAdFail = "ad_fail";
        public const string StateBlocked = "blocked";
        public const string StateGranted = "granted";
        readonly TelemetryService _telemetry;
        readonly IAdsService _ads;
        readonly AdCapPolicy _policy;
        readonly Func&lt;string&gt; _dayKey;
        public G12AdFunnel(TelemetryService telemetry, IAdsService ads, AdCapPolicy policy, Func&lt;string&gt; dayKey =
        {
            _telemetry = telemetry;
            _ads = ads;
            _policy = policy;
            _dayKey = dayKey;
        }
        /// &lt;summary&gt;Pre-offer gate: true when the point's category still has budget
        /// today (uncategorized points always pass). Null policy face = legacy
        /// ungated behavior, never a silent block.&lt;/summary&gt;
        public bool CanOffer(string pointId)
        {
            if (_policy == null) return true;
            string category = AdCategories.CategoryOfPoint(pointId);
            string key = _dayKey != null ? _dayKey() : AdCapPolicy.TodayKey();
            return _policy.HasCategoryBudget(key, category);
        }
        /// &lt;summary&gt;Accept flow entry: fires evt_ad accept, then shows the rewarded
```

```
        /// view through the category-aware service. Callers await the returned task
        /// and report granted/ad_fail from the result - an ad reward is never
        /// granted silently and never lost silently.&lt;/summary&gt;
        public Task&lt;RewardResult&gt; Accept(string pointId, string reason)
        {
            Track(StateAccept, pointId, reason);
            return _ads.ShowRewarded(pointId);
        }
        public void ReportOffer(string pointId, string reason) =&gt; Track(StateOffer, pointId, reason);
        public void ReportBlocked(string pointId, string reason) =&gt; Track(StateBlocked, pointId, reason);
        public void ReportGranted(string pointId, string reason) =&gt; Track(StateGranted, pointId, reason);
        public void ReportAdFail(string pointId, string reason) =&gt; Track(StateAdFail, pointId, reason);
        void Track(string state, string pointId, string reason)
        {
            _telemetry?.Track(EventIds.Ad, new Dictionary&lt;string, string&gt;
            {
                { "state", state },
                { "point", pointId ?? "" },
                { "reason", reason ?? "" },
                { "category", AdCategories.CategoryOfPoint(pointId) },
            });
        }
    }
}
// G12BootServices.cs
using Sy.Core;
using Sy.Platform;
using Sy.Telemetry;
using Logger = Sy.Core.Logger;
namespace Sy.G12.Boot
{
    /// &lt;summary&gt;
    /// Rewarded point ids for the three G12 ad seats (task S2-T003 numeric table,
    /// section 2 fiat): revive = the three-heart fail revive offer (rescue 4/day,
    /// E4 B5 near-miss staircase), hint = the reference-line direct answer take
    /// (direct 1/day, E4 B6 one gate per wave), ink double = the ink budget
    /// doubling offer (double 5/day, section 2 S5 extension seat). The g12_
    /// prefix keeps id namespaces isolated so no G12 call can ever draw P02/P06/
    /// G02-G08 budget and vice versa (g06_ additive paradigm, AdCategories table).
    /// &lt;/summary&gt;
    public static class G12AdPoints
    {
        public const string Revive = "g12_revive";
        public const string Hint = "g12_hint";
        public const string InkDouble = "g12_ink_double";
    }
    /// &lt;summary&gt;
    /// Platform suite wiring for G12 (task S2-T004, G02/G03/G06 BootServices
    /// paradigm). PlatformBootstrap.Create(policy) resolves editor/SY_MOCK to the
    /// mock suite and real WeChat builds to the Wx suite, so the editor path and
```

```
    /// a production build differ only by define, never by wiring. The SAME
    /// AdCapPolicy instance is exposed as SharedPolicy so offer faces pre-check
    /// budgets (HasCategoryBudget) on the exact policy the suite's ads service
    /// consumes from - pre-check and consumption can never disagree (P02 T006-B2
    /// pattern). Zero-server red line 12: the TCB batch sink class stays available
    /// in the shared package but is NEVER registered here. The Wx analytics
    /// reporter joins in task S2-T006 (G06WxAnalyticsReporter paradigm), not here.
    /// &lt;/summary&gt;
    public static class G12BootServices
    {
        /// &lt;summary&gt;The policy the registered suite consumes from; null before RegisterAll.&lt;/summary&gt;
        public static AdCapPolicy SharedPolicy { get; private set; }
        public static void RegisterAll()
        {
            if (ServiceLocator.TryGet&lt;IAdsService&gt;(out _)) return; // idempotent: double register would throw
            var policy = new AdCapPolicy();
            SharedPolicy = policy;
            var suite = PlatformBootstrap.Create(policy);
            ServiceLocator.Register&lt;IAdsService&gt;(suite.Ads);
            ServiceLocator.Register&lt;IPlatformService&gt;(suite.Platform);
            ServiceLocator.Register&lt;IStorage&gt;(suite.Storage);
            // Launch-entry query face joins the locator so a boot host can read the
            // share attribution exactly once (ILaunchInfo contract).
            ServiceLocator.Register&lt;ILaunchInfo&gt;(suite.Launch);
#if SY_WECHAT &amp;&amp; !UNITY_EDITOR
            // Wx custom analytics joins the telemetry pipeline via
            // G12TelemetryWire.CreateService (S2-T006, G06WxAnalyticsReporter
            // paradigm); mock/editor builds compile this out entirely and the
            // debug sink stays the only one.
            ServiceLocator.Register&lt;IAnalyticsReporter&gt;(new G12WxAnalyticsReporter());
#endif
            Logger.Info("BOOT", "g12 platform suite registered (suite=" + PlatformBootstrap.SuiteIdFor(suite.Channel)
                + ", g12 ad categories: rescue 4 / direct 1 / double 5, global 12/day)");
        }
    }
}
// G12TelemetryWire.cs
using Sy.Core;
using Sy.Telemetry;
using UnityEngine;
namespace Sy.G12.Boot
{
    /// &lt;summary&gt;
    /// Telemetry assembly point for G12 (task S2-T004, G06TelemetryWire paradigm):
    /// the section-8 cfg schema loads first (S2-T006, cfg_g12_events - the
    /// zero-missing alignment face), DebugLogSink is always attached; the Wx
    /// analytics sink joins when a reporter is available (explicit param wins,
    /// otherwise the locator is consulted for the boot-registered Wx reporter on
    /// real builds - G12BootServices registers G12WxAnalyticsReporter under
    /// SY_WECHAT, G06 paradigm).
```

```
    /// Zero-server red line 12: the TCB batch sink class stays available in the
    /// shared package but is never registered. Event faces: the ad funnel lives
    /// in G12AdFunnel, the six-seat section-8 family in G12TelemetryEvents.
    /// &lt;/summary&gt;
    public static class G12TelemetryWire
    {
        public static TelemetryService CreateService(IAnalyticsReporter reporter = null)
        {
            var telemetry = new TelemetryService();
            var schema = Resources.Load&lt;TextAsset&gt;("cfg_g12_events");
            if (schema != null) telemetry.LoadSchemaFromJson(schema.text);
            telemetry.AddSink(new DebugLogSink());
            if (reporter == null &amp;&amp; ServiceLocator.TryGet&lt;IAnalyticsReporter&gt;(out var fromLocator))
                reporter = fromLocator;
            if (reporter != null) telemetry.AddSink(new WxAnalyticsSink(reporter));
            return telemetry;
        }
    }
}
// G12WxAnalyticsReporter.cs
#if SY_WECHAT
using System.Runtime.InteropServices;
using Sy.Telemetry;
namespace Sy.G12.Boot
{
    /// &lt;summary&gt;
    /// WeChat analytics reporter for G12 (task S2-T006, G06WxAnalyticsReporter
    /// paradigm): bridges the telemetry WxAnalyticsSink onto the platform JS
    /// entry SyReportAnalytics declared in the shared package's SyWxBridge.jslib
    /// (com.sy.platform ships the jslib, so the extern binds on real WeChat
    /// builds with no per-project asset). Compiled only for real WeChat builds;
    /// mock/editor builds compile this out and the debug sink stays the only
    /// one. The duplicate extern declaration binds to the same jslib function
    /// the Sy.Platform Wx bridge uses.
    /// &lt;/summary&gt;
    public sealed class G12WxAnalyticsReporter : IAnalyticsReporter
    {
        [DllImport("__Internal", EntryPoint = "SyReportAnalytics")]
        static extern void ReportBridge(string evtId, string fieldsJson);
        public void Report(string evtId, string fieldsJson)
            =&gt; ReportBridge(evtId ?? "", fieldsJson ?? "{}");
    }
}
#endif
// G12AudioCatalog.cs
// G12AudioCatalog.cs - the per-project audio catalog (the G09 HospitalAudio
// X027 paradigm applied to the line-rescue clinic of G12): a ScriptableObject
// that serializes clip references by filename stem, plus the null-safe
// runtime lookup. The asset is built by the editor tool (AudioSetup) from
// Assets/_Game/Audio; runtime consumers MUST treat a missing catalog/clip
```

```
// as "skip silently" so headless checks stay green on any project state.
// Delivered stems so far: g12_bgm_main (the manifest BGM row, AU seat).
using System;
using System.Collections.Generic;
using UnityEngine;
namespace Sy.G12
{
    /// &lt;summary&gt;
    /// Generated-audio catalog (X027 paradigm): the AudioSetup editor
    /// tool scans Assets/_Game/Audio and serializes clip references
    /// here so runtime code resolves audio by filename stem. The asset
    /// lives at Assets/_Game/Resources/AudioCatalog.asset (Resources
    /// keeps runtime lookup build-safe with zero Addressables wiring).
    /// &lt;/summary&gt;
    [CreateAssetMenu(fileName = "AudioCatalog", menuName = "Sy/G12 Audio Catalog")]
    public sealed class AudioCatalog : ScriptableObject
    {
        [Serializable]
        public sealed class Entry
        {
            public string key;      // filename stem, e.g. "g12_bgm_main"
            public AudioClip clip;  // direct reference keeps builds self-contained
        }
        public List&lt;Entry&gt; entries = new List&lt;Entry&gt;();
        Dictionary&lt;string, AudioClip&gt; _index;
        void OnEnable()
        {
            // ScriptableObject instances can be revived with a stale cache;
            // drop it so the next Get() rebuilds from entries.
            _index = null;
        }
        public AudioClip Get(string key)
        {
            if (string.IsNullOrEmpty(key)) return null;
            if (_index == null)
            {
                _index = new Dictionary&lt;string, AudioClip&gt;(entries.Count, StringComparer.Ordinal);
                for (int i = 0; i &lt; entries.Count; i++)
                {
                    var e = entries[i];
                    if (e != null &amp;&amp; !string.IsNullOrEmpty(e.key) &amp;&amp; e.clip != null &amp;&amp; !_index.C
                        _index[e.key] = e.clip;
                }
            }
            return _index.TryGetValue(key, out var c) ? c : null;
        }
    }
    /// &lt;summary&gt;
    /// Runtime lookup helper. Every consumer MUST treat null as "clip
    /// missing -&gt; skip" so headless checks stay green on any project
```

```
    /// state (catalog absent, audio pruned, name typos).
    /// &lt;/summary&gt;
    public static class AudioRuntime
    {
        static AudioCatalog _catalog;
        public static AudioCatalog Catalog
        {
            get
            {
                if (_catalog == null) _catalog = Resources.Load&lt;AudioCatalog&gt;("AudioCatalog");
                return _catalog;
            }
        }
        public static AudioClip Get(string key)
        {
            var c = Catalog;
            return c != null ? c.Get(key) : null;
        }
    }
}
// G12BoardPresenter.cs
using System;
using System.Collections.Generic;
using UnityEngine;
namespace Sy.G12
{
    /// &lt;summary&gt;One presentation beat derived from a LineRescueSim event (task S1-T003).&lt;/summary&gt;
    public sealed class G12ViewCommand
    {
        public string Kind;                   // caption|guide|line_ready|hint|tension|snap|first_wow|save|fail|revive_o
        public float TimeSec;                 // sim seconds of the beat
        public float WorldX;                  // world-x anchor for located beats (snap/eraser_eat), else 0
        public Vector2 Canvas;                // canvas position for located beats, else zero
        public LineRescueSim.YLevel Level;    // line layer of located beats (default Top when not located)
        public LineRescueSim.TensionBand Band;// Snap on snap beats, Hold default
    }
    /// &lt;summary&gt;
    /// Board layout solver (task S1-T003): pure math mapping world -&amp;gt; canvas for the
    /// 1080x1920 portrait board. Kept free of MonoBehaviour/UGUI so EditMode tests
    /// can pin every corner (G03BoardLayout paradigm: the solver is the assertion
    /// oracle for the view). Heights and board bounds reference the frozen
    /// LineRescueSim constants - single source of truth, zero duplicated numbers.
    /// &lt;/summary&gt;
    public static class G12BoardLayout
    {
        public const float RefW = 1080f;
        public const float RefH = 1920f;
        public const float Scale = 200f;     // canvas px per world unit
        public const float WorldCx = 2.6f;   // playfield x 0..5.2 centered (board 0.2..5.0 + margin)
        public const float WorldCy = 3.0f;   // playfield y 0..6 centered (360px HUD margins top/bottom)
```

```
    /// &lt;summary&gt;Top/Bottom layer selector midline: halfway between the two heights.&lt;/summary&gt;
    public static float MidLevelY { get { return (LineRescueSim.TopY + LineRescueSim.BottomY) * 0.5f; } }
    public static Vector2 WorldToCanvas(float x, float y)
    {
        return new Vector2((x - WorldCx) * Scale, (y - WorldCy) * Scale);
    }
    public static Vector2 CanvasToWorld(Vector2 p)
    {
        return new Vector2(p.x / Scale + WorldCx, p.y / Scale + WorldCy);
    }
    public static float LevelY(LineRescueSim.YLevel level)
    {
        return level == LineRescueSim.YLevel.Top ? LineRescueSim.TopY : LineRescueSim.BottomY;
    }
    public static LineRescueSim.YLevel LevelFromWorldY(float y)
    {
        return y &gt;= MidLevelY ? LineRescueSim.YLevel.Top : LineRescueSim.YLevel.Bottom;
    }
    public static LineRescueSim.YLevel LevelFromCanvasY(float py)
    {
        return LevelFromWorldY(CanvasToWorld(new Vector2(0f, py)).y);
    }
    /// &lt;summary&gt;Board-x clamp for player draws: lines live inside the board bounds.&lt;/summary&gt;
    public static float ClampX(float x)
    {
        return Mathf.Clamp(x, LineRescueSim.BoardLo, LineRescueSim.BoardHi);
    }
    /// &lt;summary&gt;Canvas mid-point of a segment on its layer (snap/eat feedback anchor).&lt;/summary&gt;
    public static Vector2 SegMidCanvas(LineRescueSim.Segment seg, LineRescueSim.YLevel level)
    {
        float midX = (seg.X0 + seg.X1) * 0.5f;
        return WorldToCanvas(midX, LevelY(level));
    }
}
/// &lt;summary&gt;
/// Pure view-model for the rescue board (task S1-T003 slice A): owns the sim,
/// maps single-finger canvas input into sim draws (board clamp + tiny-span reject,
/// MaxInk clamp enforced inside the frozen sim), drains sim events into typed
/// view commands 1:1 (G03BoardPresenter paradigm), and exposes the M5 readout
/// face (hearts / ink fraction / tension bands). No MonoBehaviour and no UGUI
/// types: EditMode tests pin the command stream against the sim events. Scene
/// wiring (LineRenderer glow, input loop, HUD) lands with slices B/C.
/// &lt;/summary&gt;
public sealed class G12BoardPresenter
{
    /// &lt;summary&gt;Graybox player-line cap: the wave-one weight 1.6 snaps it on contact (first-wow hook, E4 face
    public const float DefaultDrawCap = 1.3f;
    readonly LineRescueSim _sim;
    readonly float _inkBudget;
    readonly G12InkEconomy _eco;          // T006 slice B: the M5 ledger every draw passes through
```

```
readonly float[] _starInk;            // level star thresholds (settlement verdict input)
readonly int _levelWaves;             // wave count for the level-clear face (0 = no catalog level)
readonly List&lt;G12ViewCommand&gt; _drained = new List&lt;G12ViewCommand&gt;();
readonly HashSet&lt;LineRescueSim.Segment&gt; _seenDead = new HashSet&lt;LineRescueSim.Segment&gt;();
readonly HashSet&lt;LineRescueSim.Segment&gt; _seenEaten = new HashSet&lt;LineRescueSim.Segment&gt;();
int _drainIndex;
static readonly G12ViewCommand[] Empty = new G12ViewCommand[0];
public G12BoardPresenter(LineRescueSim sim, float inkBudget)
{
    _sim = sim ?? throw new ArgumentNullException(nameof(sim));
    _inkBudget = inkBudget &gt; 0f ? inkBudget : LineRescueSim.MaxInk;
    _eco = new G12InkEconomy(_inkBudget);
}
/// &lt;summary&gt;Catalog ctor: the ink budget comes from the level table (unknown level falls back to one draw
public G12BoardPresenter(LineRescueSim sim, G12GameCatalog catalog, string levelId)
    : this(sim, BudgetOf(catalog, levelId))
{
    var level = catalog == null ? null : catalog.LevelById(levelId);
    if (level != null)
    {
        _starInk = level.starInk;
        _levelWaves = level.waveCount;
    }
}
static float BudgetOf(G12GameCatalog catalog, string levelId)
{
    var level = catalog == null ? null : catalog.LevelById(levelId);
    return level == null ? 0f : level.inkBudget;
}
public LineRescueSim Sim { get { return _sim; } }
public float InkBudget { get { return _inkBudget; } }
// --- M5 readout face (three-heart HUD / ink meter / tension bands) ---
/// &lt;summary&gt;Hearts left on the near-miss staircase (MaxFails - fails, floored at 0).&lt;/summary&gt;
public int HeartsRemaining
{
    get
    {
        int left = LineRescueSim.MaxFails - _sim.Fails;
        return left &lt; 0 ? 0 : left;
    }
}
/// &lt;summary&gt;Ink meter fraction 1..0 (T006 slice B): the economy ledger's
/// actual-span face - short strokes drain the meter proportionally.&lt;/summary&gt;
public float InkFraction()
{
    return _eco.InkFraction();
}
// --- T006 slice B: M5 economy + settlement faces ---
/// &lt;summary&gt;The ink ledger (read-only test/bot face).&lt;/summary&gt;
public G12InkEconomy Economy { get { return _eco; } }
```

```
/// &lt;summary&gt;True when the remaining ink cannot fund one more stroke.&lt;/summary&gt;
public bool PenIsDry { get { return _eco.IsDry; } }
/// &lt;summary&gt;PLAYER draws only - hint lines are paid ad seats and never enter the ledger.&lt;/summary&gt;
public int PlayerDrawCount { get { return _eco.PlayerDrawCount; } }
/// &lt;summary&gt;Level-clear face: every wave of the catalog level resolved with
/// the run still in the Playing phase (the near-miss staircase survived).&lt;/summary&gt;
public bool LevelCleared
{
    get
    {
        if (_levelWaves &lt; 1) return false;
        if (_sim.CurrentPhase != LineRescueSim.Phase.Playing) return false;
        if (_sim.WaveIndex &lt; _levelWaves) return false;
        var hs = _sim.Hazards;
        for (int i = 0; i &lt; _levelWaves &amp;&amp; i &lt; hs.Count; i++)
        {
            if (!hs[i].Resolved) return false;
        }
        return true;
    }
}
/// &lt;summary&gt;Full settlement verdict for the current run (T006): stars from
/// the level's thresholds, the one-stroke badge, ink used, and the dry-fail
/// flag that feeds the M4 revive staircase.&lt;/summary&gt;
public G12InkEconomy.Settlement Settle()
{
    return G12InkEconomy.Settle(LevelCleared, _eco, _starInk);
}
/// &lt;summary&gt;Tension-band mirror of the frozen sim classifier (one shared truth for view feedback).&lt;/su
public static LineRescueSim.TensionBand BandFor(float load)
{
    return LineRescueSim.Classify(load);
}
/// &lt;summary&gt;Worst live tension band across all loaded segments (per-frame glow source for slice C).&lt;/s
public LineRescueSim.TensionBand HotBand()
{
    var worst = LineRescueSim.TensionBand.Hold;
    var lines = _sim.Lines;
    for (int i = 0; i &lt; lines.Count; i++)
    {
        var segs = lines[i].Segs;
        for (int s = 0; s &lt; segs.Count; s++)
        {
            var seg = segs[s];
            if (!seg.Live || seg.Load &lt;= 0f) continue;
            var band = LineRescueSim.Classify(seg.Load);
            if (band == LineRescueSim.TensionBand.Snap) return LineRescueSim.TensionBand.Snap;
            if (band == LineRescueSim.TensionBand.NearSnap) worst = LineRescueSim.TensionBand.NearSnap;
        }
    }
```

```
    return worst;
}
// --- S2-T007 slice B: share-run capture ---
// Every accepted player commit (draw or granted hint) logs its sim-clock
// beat in production order; the share encoder replays exactly this log
// through the same commit law, so the shipped code is the proof of the run.
readonly System.Collections.Generic.List&lt;G12ShareStroke&gt; _shareLog =
    new System.Collections.Generic.List&lt;G12ShareStroke&gt;();
/// &lt;summary&gt;Snapshot of the recorded run for the share encoder
/// (task S2-T007): the live log is never handed out.&lt;/summary&gt;
public G12ShareStroke[] ShareLogSnapshot() { return _shareLog.ToArray(); }
// --- input mapping (single-finger draw: canvas -&gt; world -&gt; sim) ---
/// &lt;summary&gt;
/// Player draw: two canvas points become one board line. X endpoints clamp to
/// the board bounds, the layer follows the stroke mid-height, tiny spans are
/// rejected by the sim rule, and the sim enforces the MaxInk clamp. Returns
/// the drained commands - empty when the draw was rejected.
/// &lt;/summary&gt;
public G12ViewCommand[] DrawFromCanvas(Vector2 canvasA, Vector2 canvasB, float capPerSeg = DefaultDrawCap)
{
    var a = G12BoardLayout.CanvasToWorld(canvasA);
    var b = G12BoardLayout.CanvasToWorld(canvasB);
    float x0 = G12BoardLayout.ClampX(Mathf.Min(a.x, b.x));
    float x1 = G12BoardLayout.ClampX(Mathf.Max(a.x, b.x));
    var level = G12BoardLayout.LevelFromWorldY((a.y + b.y) * 0.5f);
    // T006 slice B M5 ink gate: a stroke that does not fit the remaining
    // ink draws nothing (a dry pen never commits a phantom line). The
    // ledger charges the same clamped span the sim will draw, and both
    // tiny-span rejects share the 0.05 threshold, so an accepted stroke
    // always clears the sim rule.
    if (!_eco.TryConsumeDraw(x1 - x0)) return Empty;
    if (!_sim.DrawLine(x0, x1, level, capPerSeg)) return Empty;
    // S2-T007 slice B: only accepted commits ride the share log - the
    // replay re-runs the same gates on a fresh economy, so an accepted
    // stroke here is an accepted stroke there (byte-stable recompute).
    _shareLog.Add(new G12ShareStroke
    {
        TimeSec = _sim.Clock,
        X0 = x0,
        X1 = x1,
        TopLayer = level == LineRescueSim.YLevel.Top,
    });
    return Drain();
}
// --- pass-through faces (each drains whatever fired) ---
public G12ViewCommand[] Open() { _sim.Open(); return Drain(); }
public G12ViewCommand[] MaybeShowGuide() { _sim.MaybeShowGuide(); return Drain(); }
/// &lt;summary&gt;Read-only hint-seat probe (S2-T004): true when the hint seat
/// could deliver a reference line right now - the view pre-checks before
/// firing the rewarded view so a grant is never wasted on a denied gate
```

```
/// (never-waste law, mirror of the sim probe contract).&lt;/summary&gt;
public bool HintAvailableThisWave { get { return _sim.HintAvailableThisWave; } }
public G12ViewCommand[] RequestHint()
{
    // S2-T007 slice B: a granted hint beat rides the share log too -
    // the replay re-derives the ghost line at the same beat.
    if (_sim.HintAvailableThisWave)
        _shareLog.Add(new G12ShareStroke { TimeSec = _sim.Clock, IsHint = true });
    _sim.RequestHint();
    return Drain();
}
public G12ViewCommand[] AcceptRevive() { _sim.AcceptRevive(); return Drain(); }
public G12ViewCommand[] AcceptRetry() { _sim.AcceptRetry(); return Drain(); }
public G12ViewCommand[] Advance(float dt) { _sim.AdvanceClock(dt); return Drain(); }
/// &lt;summary&gt;
/// Drains every unprocessed sim event into exactly one view command (1:1 parity:
/// same count, same order, same sim seconds). Snap and eraser-eat beats carry
/// the canvas position of the segment they killed - those state transitions
/// leave a unique fingerprint, so the located anchor is deterministic.
/// &lt;/summary&gt;
public G12ViewCommand[] Drain()
{
    _drained.Clear();
    var events = _sim.Events;
    for (int i = _drainIndex; i &lt; events.Count; i++)
    {
        var e = events[i];
        var cmd = new G12ViewCommand { Kind = MapKind(e.Kind), TimeSec = e.SimSeconds };
        if (e.Kind == "line_snap")
        {
            cmd.Band = LineRescueSim.TensionBand.Snap;
            Locate(cmd, true);
        }
        else if (e.Kind == "eraser_eat")
        {
            Locate(cmd, false);
        }
        _drained.Add(cmd);
    }
    _drainIndex = events.Count;
    return _drained.ToArray();
}
void Locate(G12ViewCommand cmd, bool dead)
{
    var seen = dead ? _seenDead : _seenEaten;
    var lines = _sim.Lines;
    for (int i = lines.Count - 1; i &gt;= 0; i--)   // newest line first
    {
        var line = lines[i];
        var segs = line.Segs;
```

```
                for (int s = 0; s &lt; segs.Count; s++)
                {
                    var seg = segs[s];
                    bool hit = dead ? (seg.Dead &amp;&amp; !seen.Contains(seg)) : (seg.Eaten &amp;&amp; !seen.Contains(s
                    if (!hit) continue;
                    seen.Add(seg);
                    cmd.WorldX = (seg.X0 + seg.X1) * 0.5f;
                    cmd.Level = line.Level;
                    cmd.Canvas = G12BoardLayout.SegMidCanvas(seg, line.Level);
                    return;
                }
            }
        }
        static string MapKind(string kind)
        {
            switch (kind)
            {
                case "caption": return "caption";
                case "guide_prompt": return "guide";
                case "line_ready": return "line_ready";
                case "line_tension": return "tension";
                case "line_snap": return "snap";
                case "first_wow": return "first_wow";
                case "rescue_success": return "save";
                case "rescue_fail": return "fail";
                case "revive_offer": return "revive_offer";
                case "free_retry": return "free_retry";   // v1.1: the beat keeps its kind (drain is 1:1)
                case "hint_line_shown": return "hint";
                case "eraser_spawn": return "eraser_spawn";
                case "eraser_eat": return "eraser_eat";
                default: return "board";   // unknown beat: board resync
            }
        }
    }
}
// G12CreepFocus.cs
using UnityEngine;
namespace Sy.G12
{
    /// &lt;summary&gt;
    /// Creep-focus flow table for task S3-T004 slice B (X022 per-title polish):
    /// the NearSnap creep lands as a held-breath close-up - an inset magnifier
    /// panel armed on the creep rising edge (a live run entering the NearSnap
    /// band), held through the verdict. When the line snaps, the frozen 0.55s
    /// collapse slow-mo plays INSIDE the inset (the S1-T004/T005 beat language
    /// re-checked here: the episode hold equals the slow-mo window, so the
    /// strain and the collapse read as one beat); when the strain eases, the
    /// inset retires with it. Pure constants + pure math only (G12SettleFlow /
    /// G12HookFlow sibling paradigm): the frozen sim is never touched (E4 13
    /// anchors, the [0.55, 0.90] calibration window C156 stay byte-identical),
```

```
/// the main board never moves (the 1:1 world-canvas mapping, the settlement
/// banner C156 and the revive offer C150 same-frame anchors are untouched -
/// the close-up is a view-only second camera on a render texture), and this
/// table adds zero new color faces (the frame reuses the NearSnap amber, so
/// the ten-face distinctness law keeps holding). EditMode tests pin the
/// whole timeline without a scene.
/// &lt;/summary&gt;
public static class G12CreepFocus
{
    // --- episode windows (view beats; mirrors of frozen faces, zero new tuning) ---
    /// &lt;summary&gt;Fastest band-entry to snap verdict (E4 B2 frozen creep window low bound).&lt;/summary&gt;
    public const float CreepWindowLoSec = 0.20f;
    /// &lt;summary&gt;Slowest creep verdict (E4 B2 frozen creep window high bound).&lt;/summary&gt;
    public const float CreepWindowHiSec = 0.45f;
    /// &lt;summary&gt;The inset must be fully staged before the fastest verdict can land.&lt;/summary&gt;
    public const float ApproachSec = 0.15f;
    /// &lt;summary&gt;Ease-out retire window (the relief lift reads snappy, never lingers).&lt;/summary&gt;
    public const float ReleaseSec = 0.20f;
    /// &lt;summary&gt;Worst-case episode budget cap (slowest verdict + snap hold + retire).&lt;/summary&gt;
    public const float EpisodeBudgetCapSec = 1.30f;
    // --- inset face (canvas geometry: left-upper free zone, clear of every
    // staged UI row - hearts top-left, quip 744, guide 640, erase toast 460) ---
    public const float PanelAnchorX = -330f;
    public const float PanelAnchorY = 150f;
    public const float PanelSizeCanvas = 360f;
    public const float PanelFramePad = 10f;
    public const int RtSize = 512;
    /// &lt;summary&gt;Tight close-up half-height in world units (the strained run + adjacent board).&lt;/summary&gt
    public const float InsetOrthoSize = 1.2f;
    /// &lt;summary&gt;The erase-toast row the inset must never cover (seat anchored at y 460).&lt;/summary&gt;
    public const float EraseToastRowY = 460f;
    /// &lt;summary&gt;Episode phase (pure state machine; verdict time &amp;lt;= 0 means the episode is still open).
    public enum Phase { Idle, Approach, Hold, SnapHold, Release }
    /// &lt;summary&gt;The snap hold equals the frozen collapse slow-mo window (the 0.55s beat re-check).&lt;/summar
    public static float SnapHoldExtraSec() { return G12StageStyle.SlowmoDurationSec; }
    /// &lt;summary&gt;S1 beat-language re-check (the slice-B review face): the inset must hold
    /// through the whole frozen snap slow-mo - the collapse plays inside the
    /// close-up, one held beat from strain to verdict.&lt;/summary&gt;
    public static bool SlowmoRecheckPasses()
    {
        return G12StageStyle.SlowmoWithinBudget()
            &amp;&amp; SnapHoldExtraSec() == G12StageStyle.SlowmoDurationSec;
    }
    /// &lt;summary&gt;True when the approach lands before the fastest frozen creep verdict.&lt;/summary&gt;
    public static bool ApproachLandsInWindow() { return ApproachSec &lt; CreepWindowLoSec; }
    /// &lt;summary&gt;Worst-case episode length (slowest verdict + snap hold + retire).&lt;/summary&gt;
    public static float EpisodeBudgetMaxSec()
    {
        return CreepWindowHiSec + SnapHoldExtraSec() + ReleaseSec;
    }
```

```
/// &lt;summary&gt;True when the worst episode still reads as one beat (never lingers).&lt;/summary&gt;
public static bool EpisodeWithinBudget()
{
    return EpisodeBudgetMaxSec() &lt;= EpisodeBudgetCapSec;
}
/// &lt;summary&gt;Panel top edge in canvas units (the inset's own footprint).&lt;/summary&gt;
public static float PanelTopY()
{
    return PanelAnchorY + (PanelSizeCanvas + PanelFramePad * 2f) * 0.5f;
}
/// &lt;summary&gt;True when the inset never covers the eraser toast row (seat law).&lt;/summary&gt;
public static bool PanelClearsToastRow()
{
    return PanelTopY() &lt; EraseToastRowY;
}
// --- panel scale curve (pop in, hold, retire) ---
/// &lt;summary&gt;Pop-in ease 0..1 over the approach window (ease-out cubic: fast
/// arrival, gentle settle; same input, same output).&lt;/summary&gt;
public static float PopEase(float sec)
{
    if (sec &lt;= 0f) return 0f;
    if (sec &gt;= ApproachSec) return 1f;
    float k = sec / ApproachSec;
    return 1f - (1f - k) * (1f - k) * (1f - k);
}
/// &lt;summary&gt;Retire ease 1..0 for a release fraction 0..1 (quadratic fade).&lt;/summary&gt;
public static float RetireEase(float frac)
{
    float k = frac &lt; 0f ? 0f : (frac &gt; 1f ? 1f : frac);
    return (1f - k) * (1f - k);
}
/// &lt;summary&gt;Panel scale at an episode time (pure): pops in over the approach,
/// holds at 1 through the creep and the snap slow-mo window, eases out
/// over the retire window, 0 after. verdictAtSec &amp;lt;= 0 keeps the episode
/// open (approach/hold only). The release-edge check carries a tiny
/// epsilon: (0.50f - 0.30f) / 0.20f lands one float ulp below 1, and the
/// panel must read exactly retired at the window edge, never 3.55e-15
/// (a zero-scale panel lingering one frame is a real retire jank, not
/// just a test nicety).&lt;/summary&gt;
public static float PanelScaleAt(float elapsedSec, float verdictAtSec, bool snapped)
{
    if (elapsedSec &lt;= 0f) return 0f;
    bool verdict = verdictAtSec &gt; 0f &amp;&amp; elapsedSec &gt;= verdictAtSec;
    if (!verdict) return PopEase(elapsedSec);
    if (snapped &amp;&amp; elapsedSec &lt; verdictAtSec + SnapHoldExtraSec()) return 1f;
    float releaseStart = snapped ? verdictAtSec + SnapHoldExtraSec() : verdictAtSec;
    float popAtVerdict = PopEase(verdictAtSec &lt; ApproachSec ? verdictAtSec : ApproachSec);
    float t = (elapsedSec - releaseStart) / ReleaseSec;
    if (t &lt;= 0f) return popAtVerdict;
    if (t &gt;= 1f - 1e-5f) return 0f;
```

```
            return popAtVerdict * RetireEase(t);
        }
        /// &lt;summary&gt;Episode phase at a time (pure state machine, mirrors PanelScaleAt).&lt;/summary&gt;
        public static Phase PhaseAt(float elapsedSec, float verdictAtSec, bool snapped)
        {
            if (elapsedSec &lt; 0f) return Phase.Idle;
            bool verdict = verdictAtSec &gt; 0f &amp;&amp; elapsedSec &gt;= verdictAtSec;
            if (!verdict) return elapsedSec &lt; ApproachSec ? Phase.Approach : Phase.Hold;
            if (snapped &amp;&amp; elapsedSec &lt; verdictAtSec + SnapHoldExtraSec()) return Phase.SnapHold;
            float releaseStart = snapped ? verdictAtSec + SnapHoldExtraSec() : verdictAtSec;
            if (elapsedSec &lt; releaseStart + ReleaseSec) return Phase.Release;
            return Phase.Idle;
        }
        // --- inset camera face (view-only second camera; the main board never moves) ---
        /// &lt;summary&gt;Inset camera position for a focus anchor (fixed depth behind the board).&lt;/summary&gt;
        public static Vector3 InsetCamPosAt(Vector2 anchorWorld)
        {
            return new Vector3(anchorWorld.x, anchorWorld.y, -10f);
        }
        /// &lt;summary&gt;Inset orthographic half-height (tight close-up, frozen constant).&lt;/summary&gt;
        public static float InsetOrtho() { return InsetOrthoSize; }
    }
}
// G12EconomyCatalog.cs
using System;
using System.Collections.Generic;
using UnityEngine;
using Logger = Sy.Core.Logger;
namespace Sy.G12
{
    [Serializable]
    public sealed class AdPointData
    {
        public string id;         // g12_-namespaced rewarded point id
        public string category;   // shared budget pool: rescue / direct / double
        public int dailyCap;      // per-category daily cap (requirement doc section 2 fiat)
        public string seat;       // S2-T004 wiring face: revive / hint / double
        public string nameKey;    // i18n coverage key
    }
    [Serializable]
    public sealed class EconomyFile
    {
        public int globalCap;
        public int pointCountCap;
        public AdPointData[] points;
    }
    /// &lt;summary&gt;
    /// Static ad-economy table for LineRescue (task S2-T003, numeric landing). The
    /// section-2 fiat (rescue 4 / direct 1 / double 5 per day, ad points capped at
    /// seven, global cap 12) lives here as data; the shared Sy.Platform AdCapPolicy
```

```
/// owns the same numbers, so Validate() asserts the two sources never drift:
/// every point id must map to its declared category and every declared cap must
/// equal the policy cap (dual-source zero-drift). Unknown lookups return null
/// and are never an error (G12GameCatalog paradigm).
/// &lt;/summary&gt;
public sealed class G12EconomyCatalog
{
    public const string SeatRevive = "revive";
    public const string SeatHint = "hint";
    public const string SeatDouble = "double";
    public int GlobalCap { get; }
    public int PointCountCap { get; }
    public AdPointData[] Points { get; }
    G12EconomyCatalog(EconomyFile file)
    {
        GlobalCap = file.globalCap;
        PointCountCap = file.pointCountCap;
        Points = file.points;
    }
    public static G12EconomyCatalog Load()
    {
        var asset = Resources.Load&lt;TextAsset&gt;("cfg_g12_economy");
        if (asset == null)
        {
            Logger.Warn("G12.Economy", "cfg_g12_economy.json not found under Resources");
            return null;
        }
        var file = JsonUtility.FromJson&lt;EconomyFile&gt;(asset.text);
        if (file == null || file.points == null || file.points.Length == 0)
        {
            Logger.Warn("G12.Economy", "cfg_g12_economy.json failed to parse");
            return null;
        }
        return new G12EconomyCatalog(file);
    }
    /// &lt;summary&gt;
    /// Unknown point ids (and empty ids) return null - callers must treat that
    /// as "not configured", never as an error.
    /// &lt;/summary&gt;
    public AdPointData PointById(string id)
    {
        if (string.IsNullOrEmpty(id)) return null;
        foreach (var p in Points)
        {
            if (p.id == id) return p;
        }
        return null;
    }
    // -- dual-source accessors: the shared policy table is the second source;
    //    these expose it so tests and gates can assert zero drift without
```

```
//    referencing Sy.Platform themselves.
/// &lt;summary&gt;Category the shared policy maps this point id to (null = unmapped).&lt;/summary&gt;
public static string MappedCategoryOf(string pointId)
    =&gt; Sy.Platform.AdCategories.CategoryOfPoint(pointId);
/// &lt;summary&gt;Daily cap the shared policy holds for a category (0 = uncategorized).&lt;/summary&gt;
public static int PolicyCategoryCap(string category)
    =&gt; Sy.Platform.AdCapPolicy.CategoryCap(category);
/// &lt;summary&gt;The policy's global rewarded cap (the 12/day hard line).&lt;/summary&gt;
public static int PolicyGlobalCap =&gt; Sy.Platform.AdCapPolicy.DefaultDailyCap;
/// &lt;summary&gt;
/// Structural validation shared by the EditMode tests and the CoreCheck gate.
/// Returns the list of problems; empty means the table is well-formed and
/// aligned with the shared policy (both sources agree on every value).
/// &lt;/summary&gt;
public List&lt;string&gt; Validate()
{
    var problems = new List&lt;string&gt;();
    if (GlobalCap != PolicyGlobalCap)
        problems.Add("globalCap must stay at the policy cap " + PolicyGlobalCap + ", got " + GlobalCap);
    if (PointCountCap &lt; 1 || PointCountCap &gt; 7)
        problems.Add("pointCountCap out of 1..7 (section 2 ad-point ceiling), got " + PointCountCap);
    if (Points.Length &gt; PointCountCap)
        problems.Add("declared points " + Points.Length + " exceed the pointCountCap " + PointCountCap);
    var ids = new HashSet&lt;string&gt;();
    var seats = new HashSet&lt;string&gt;();
    for (int i = 0; i &lt; Points.Length; i++)
    {
        var p = Points[i];
        string tag = "point[" + i + "]";
        if (string.IsNullOrEmpty(p.id)) problems.Add(tag + " missing id");
        else if (!ids.Add(p.id)) problems.Add(tag + " duplicate id " + p.id);
        else if (!p.id.StartsWith("g12_", StringComparison.Ordinal))
            problems.Add(tag + " id must stay g12_-namespaced, got " + p.id);
        if (string.IsNullOrEmpty(p.category))
            problems.Add(tag + " missing category");
        else
        {
            int cap = PolicyCategoryCap(p.category);
            if (cap &lt;= 0) problems.Add(tag + " category unknown to the policy: " + p.category);
            else if (p.dailyCap != cap)
                problems.Add(tag + " dailyCap " + p.dailyCap + " drifts from policy cap " + cap + " for " + p.ca
        }
        string mapped = MappedCategoryOf(p.id);
        if (mapped == null) problems.Add(tag + " unmapped in the shared policy table: " + p.id);
        else if (mapped != p.category)
            problems.Add(tag + " mapped category " + mapped + " drifts from declared " + p.category);
        if (p.dailyCap &lt; 1 || p.dailyCap &gt; 12)
            problems.Add(tag + " dailyCap out of 1..12, got " + p.dailyCap);
        if (p.seat != SeatRevive &amp;&amp; p.seat != SeatHint &amp;&amp; p.seat != SeatDouble)
            problems.Add(tag + " seat must be revive/hint/double, got " + p.seat);
```

```
                else if (!seats.Add(p.seat))
                    problems.Add(tag + " duplicate seat " + p.seat);
                if (string.IsNullOrEmpty(p.nameKey))
                    problems.Add(tag + " nameKey missing");
            }
            // Section-2 completeness: all three seats stay declared (S3 revive /
            // S4 hint / S5 ink double) - the S2-T004 wiring face reads them all.
            if (!seats.Contains(SeatRevive)) problems.Add("seat table incomplete: revive seat missing (section 2 S3)");
            if (!seats.Contains(SeatHint)) problems.Add("seat table incomplete: hint seat missing (section 2 S4)");
            if (!seats.Contains(SeatDouble)) problems.Add("seat table incomplete: double seat missing (section 2 S5)");
            return problems;
        }
    }
}
// G12GameCatalog.cs
using System;
using System.Collections.Generic;
using UnityEngine;
using Logger = Sy.Core.Logger;
namespace Sy.G12
{
    [Serializable]
    public sealed class LevelData
    {
        public string id;              // L01..L10
        public string terrain;         // "double" (both anchor YLevels) / "top" / "bottom"
        public int waveCount;          // hazard waves in this level
        public float waveIntervalSec;   // 0 = inherit the frozen LineRescueSim.WaveInterval
        public int eraserCount;        // eraser appearances in this level (0 = none)
        public int eraserStartWave;    // 1-based wave index of the first eraser (0 when eraserCount = 0)
        public int hearts;             // must stay at LineRescueSim.MaxFails (near-miss staircase, frozen parity)
        public float inkBudget; // total ink for the level (M5 economy face, quarter-draw grid 0.5..4 MaxInk)
        public float[] starInk;        // ink-used thresholds [1-star, 2-star, 3-star], shorter draw = more stars
        public string nameKey;         // i18n key for the display name
    }
    [Serializable]
    public sealed class LevelsFile
    {
        public int chapter;
        public string chapterKey;
        public LevelData[] levels;
    }
    /// &lt;summary&gt;
    /// Static level catalog for LineRescue (task S1-T002). Data layer only: chapter 1
    /// carries the ten graybox-tier levels. The table feeds the S1-T003 view wiring and
    /// the T006 ink economy; unknown lookups return null and are never an error
    /// (G03GameCatalog paradigm). Parity anchors are enforced by Validate():
    /// hearts stay at MaxFails, ink budgets sit on the quarter-draw grid
    /// (whole-draw placeholder retired by the C156 sweep calibration), and the first
    /// level teaches drawing before any eraser threat (red line 11 teaching curve).
```

```
/// &lt;/summary&gt;
public sealed class G12GameCatalog
{
    public const string TerrainDouble = "double";
    public const string TerrainTop = "top";
    public const string TerrainBottom = "bottom";
    public int Chapter { get; }
    public string ChapterKey { get; }
    public LevelData[] Levels { get; }
    G12GameCatalog(LevelsFile file)
    {
        Chapter = file.chapter;
        ChapterKey = file.chapterKey;
        Levels = file.levels;
    }
    public static G12GameCatalog Load()
    {
        var asset = Resources.Load&lt;TextAsset&gt;("cfg_g12_levels");
        if (asset == null)
        {
            Logger.Warn("G12.Catalog", "cfg_g12_levels.json not found under Resources");
            return null;
        }
        var file = JsonUtility.FromJson&lt;LevelsFile&gt;(asset.text);
        if (file == null || file.levels == null || file.levels.Length == 0)
        {
            Logger.Warn("G12.Catalog", "cfg_g12_levels.json failed to parse");
            return null;
        }
        return new G12GameCatalog(file);
    }
    /// &lt;summary&gt;
    /// Unknown level ids (and empty ids) return null - callers must treat that
    /// as "not configured", never as an error.
    /// &lt;/summary&gt;
    public LevelData LevelById(string id)
    {
        if (string.IsNullOrEmpty(id)) return null;
        foreach (var l in Levels)
        {
            if (l.id == id) return l;
        }
        return null;
    }
    /// &lt;summary&gt;
    /// Structural validation shared by the EditMode tests and any future gate.
    /// Returns the list of problems; empty means the table is well-formed.
    /// &lt;/summary&gt;
    public List&lt;string&gt; Validate()
    {
```

```
var problems = new List&lt;string&gt;();
if (Chapter &lt; 1) problems.Add("chapter must be &gt;= 1");
if (string.IsNullOrEmpty(ChapterKey)) problems.Add("chapterKey missing");
var ids = new HashSet&lt;string&gt;();
for (int i = 0; i &lt; Levels.Length; i++)
{
    var l = Levels[i];
    string tag = "level[" + i + "]";
    if (string.IsNullOrEmpty(l.id)) problems.Add(tag + " missing id");
    else if (!ids.Add(l.id)) problems.Add(tag + " duplicate id " + l.id);
    if (l.terrain != TerrainDouble &amp;&amp; l.terrain != TerrainTop &amp;&amp; l.terrain != TerrainBottom)
        problems.Add(tag + " terrain must be double/top/bottom, got " + l.terrain);
    if (l.waveCount &lt; 1 || l.waveCount &gt; 12)
        problems.Add(tag + " waveCount out of 1..12, got " + l.waveCount);
    if (l.hearts != LineRescueSim.MaxFails)
        problems.Add(tag + " hearts must stay at MaxFails " + LineRescueSim.MaxFails + " (graybox parity), g
    if (!InkBudgetIsOnQuarterDrawGrid(l.inkBudget))
        problems.Add(tag + " inkBudget must sit on the quarter-draw grid (0.5..4 MaxInk draws, C156 sweep ca
    if (l.starInk == null || l.starInk.Length != 3)
        problems.Add(tag + " starInk must hold exactly 3 thresholds");
    else
    {
        if (l.starInk[0] &lt;= l.starInk[1] || l.starInk[1] &lt;= l.starInk[2] || l.starInk[2] &lt;= 0f)
            problems.Add(tag + " starInk must be descending and positive (shorter draw = more stars)");
        if (l.starInk[0] &gt; l.inkBudget)
            problems.Add(tag + " 1-star ink threshold exceeds the level ink budget");
    }
    bool noEraser = l.eraserCount == 0;
    bool noStart = l.eraserStartWave == 0;
    if (noEraser != noStart)
        problems.Add(tag + " eraserCount/eraserStartWave must be paired (both zero or both set)");
    if (l.eraserCount &lt; 0 || l.eraserCount &gt; 2)
        problems.Add(tag + " eraserCount out of 0..2, got " + l.eraserCount);
    if (l.eraserCount &gt; 0)
    {
        if (l.eraserStartWave &lt; 1 || l.eraserStartWave &gt; l.waveCount)
            problems.Add(tag + " eraserStartWave out of 1..waveCount, got " + l.eraserStartWave);
        if (l.eraserCount &gt; l.waveCount)
            problems.Add(tag + " eraserCount exceeds waveCount");
    }
    if (l.waveIntervalSec != 0f
        &amp;&amp; (l.waveIntervalSec &lt; LineRescueSim.WaveInterval * 0.5f
            || l.waveIntervalSec &gt; LineRescueSim.WaveInterval * 2f))
        problems.Add(tag + " waveIntervalSec must be 0 (frozen default) or within 0.5x..2x of " + LineRescue
    if (string.IsNullOrEmpty(l.nameKey))
        problems.Add(tag + " nameKey missing");
}
// Teaching-curve guard: the very first level never opens with an eraser
// threat - the draw verb is taught clean before the defense pressure
// starts (red line 11, E4 hook face order).
```

```
            if (Levels.Length &gt; 0 &amp;&amp; Levels[0].eraserCount != 0)
                problems.Add("first level must teach drawing without an eraser threat");
            return problems;
        }
        /// &lt;summary&gt;Budget grid rule (C156 calibration sync): the sweep axis quantizes
        /// budgets to quarter MaxInk draws in 0.5..4 draws. The old whole-draw rule
        /// was the graybox placeholder - the first honest sweep proved early levels
        /// need sub-draw budgets (L01 clears at 1.8 ink), so the grid extends DOWN
        /// with the evidence on record (ink_sweep_report.json + task book C156 row).&lt;/summary&gt;
        static bool InkBudgetIsOnQuarterDrawGrid(float budget)
        {
            if (budget &lt;= 0f) return false;
            float quarters = budget / (LineRescueSim.MaxInk * 0.25f);
            int whole = Mathf.RoundToInt(quarters);
            if (whole &lt; 2 || whole &gt; 16) return false;   // 0.5..4 draws
            return Mathf.Abs(quarters - whole) &lt; 0.01f;
        }
    }
}
// G12HookFlow.cs
using UnityEngine;
namespace Sy.G12
{
    /// &lt;summary&gt;
    /// Ten-second hook staging table (task S1-T005, red line 11: docs/04 v0 is the
    /// threshold source). The beat truths already live in the frozen sim (caption
    /// 0.00s = H1, first-draw guide 0.05s = H2, first-line same-tick = H3, three
    /// rescues inside 10s = H4, wave-one weight 1.6 snapping the first line = H5);
    /// this table stages the visible drama of the T+0-3s hook picture on top of those
    /// frozen beats: the cliff dog trembles while the level plays (alive and scared
    /// before any draw), the lava ember rises through the hook window then holds the
    /// full threat face, and every snap beat lands as a decaying camera punch that
    /// rides inside the T004 collapse slow-mo so the first thrill reads as ONE beat.
    /// Pure constants + pure math over frozen sim clocks only - the sim itself is
    /// never touched, so EditMode tests pin the whole hook timeline without a scene
    /// (G12StageStyle sibling).
    /// &lt;/summary&gt;
    public static class G12HookFlow
    {
        // T+0-3s hook window (docs/04: the hook picture must read inside 3s). The
        // lava approach saturates at the window edge and never fades after.
        public const float HookWindowSec = 3.0f;
        // victim tremble (the cliff dog): a small positional shiver so the stranded
        // one reads as alive and scared before the first draw. The amplitude stays
        // well under the fail-line band so the tremble never reads as movement.
        public const float TrembleCadenceHz = 6.5f;
        public const float TrembleAmpWorld = 0.022f;
        /// &lt;summary&gt;Deterministic tremble offset for a render time (same input, same offset).&lt;/summary&gt;
        public static float TrembleOffset(float timeSec)
        {
```

```
            return TrembleAmpWorld * Mathf.Sin(Mathf.PI * 2f * TrembleCadenceHz * timeSec);
        }
        // lava approach (the threat under the cliff): the ember glow rises 0 -&gt; 1
        // through the hook window, then holds. Deep ember is its own face - it must
        // never read as a snap collapse, an eraser bite, the offer gold or a band.
        public const int LavaGlowHex = 0xE25822;
        /// &lt;summary&gt;Ember level 0..1 for a sim clock: rises through the hook window, saturates after.&lt;/summary
        public static float LavaGlowLevel(float clockSec)
        {
            if (clockSec &lt; 0f) return 0f;
            float k = clockSec / HookWindowSec;
            return k &gt; 1f ? 1f : k;
        }
        // first-snap punch (GDD: the first snap = screen shake + sfx first-thrill;
        // the sfx axis is the next slice): a short decaying camera shake whose
        // window rides inside the T004 collapse slow-mo (0.55s) so the punch and
        // the budget-window collapse read as ONE beat, never two.
        public const float SnapShakeDurationSec = 0.22f;
        public const float SnapShakeAmpWorld = 0.05f;
        public const float SnapShakeYScale = 0.6f;   // vertical share: asymmetry reads organic
        /// &lt;summary&gt;Deterministic shake offset for a remaining window (same input, same offset).
        /// The offset fades linearly to rest at the window edge - the punch decays, never cuts.&lt;/summary&gt;
        public static Vector2 SnapShakeOffset(float windowSec)
        {
            if (windowSec &lt;= 0f || windowSec &gt; SnapShakeDurationSec) return Vector2.zero;
            float t = SnapShakeDurationSec - windowSec;   // window counts down; the phase runs forward
            const float w = Mathf.PI * 2f * 16f;
            float fade = windowSec / SnapShakeDurationSec;
            return new Vector2(
                SnapShakeAmpWorld * Mathf.Sin(w * t) * fade,
                SnapShakeAmpWorld * SnapShakeYScale * Mathf.Cos(w * 1.37f * t) * fade);
        }
    }
}
// G12HudStyle.cs
using System.Collections.Generic;
using UnityEngine;
namespace Sy.G12
{
    /// &lt;summary&gt;Visual parameters for one tension band (task S1-T003 slice C): the
    /// line face escalates Hold, NearSnap, Snap so the creep reads before the snap.&lt;/summary&gt;
    public struct G12BandStyle
    {
        public int Hex;            // line color for the band
        public float WidthMul;     // LineRenderer width multiplier (pulse base)
        public float PulsePerSec;  // width breathing rate (0 = steady)
    }
    /// &lt;summary&gt;
    /// Pure HUD style + geometry for task S1-T003 slice C (G03 style-table paradigm):
    /// three-heart row, ink meter anchors and the tension-band visual table. Everything
```

```
/// is static data or pure math over the frozen sim read-only faces, so EditMode
/// tests pin the whole table without a scene. Color rationale: the accent stays the
/// GDD-frozen neon marker green; the warm escalation (amber band, pink hearts) is
/// functional feedback - lives and rising tension must read at a glance on a
/// green-ink board (filter 0: readability first).
/// &lt;/summary&gt;
public static class G12HudStyle
{
    // three-heart row: top-left lives convention, rightmost heart drains first
    public const int HeartSlots = LineRescueSim.MaxFails;
    public const float HeartSize = 56f;
    public const float HeartSpacing = 96f;
    public static readonly Vector2 HeartRowAnchor = new Vector2(-420f, 896f);
    public const int HeartFilledHex = 0xFF5A7A;   // neon pink lives (warm code reads against green ink)
    public const int HeartLostHex = 0x3A4652;     // dim slate corpse
    // ink meter: bottom strip (the ink IS the marker green)
    public const float InkBarW = 560f;
    public const float InkBarH = 26f;
    public static readonly Vector2 InkBarAnchor = new Vector2(0f, -880f);
    public const float InkLabelPad = 18f;
    public const int InkBarBgHex = 0x22303C;
    /// &lt;summary&gt;Slot anchor left-to-right (slot 0 is leftmost).&lt;/summary&gt;
    public static Vector2 HeartSlotAnchor(int slot)
    {
        return new Vector2(HeartRowAnchor.x + slot * HeartSpacing, HeartRowAnchor.y);
    }
    /// &lt;summary&gt;Pure slot fill map: slots below the remaining count stay lit.&lt;/summary&gt;
    public static bool HeartFilled(int slot, int heartsRemaining)
    {
        return slot &lt; heartsRemaining;
    }
    /// &lt;summary&gt;Ink fill width for a fraction (clamped): the meter never draws backwards.&lt;/summary&gt;
    public static float InkFillWidth(float fraction)
    {
        float f = fraction &lt; 0f ? 0f : (fraction &gt; 1f ? 1f : fraction);
        return InkBarW * f;
    }
    /// &lt;summary&gt;Tension-band visual table: the single source the view reads per run.&lt;/summary&gt;
    public static G12BandStyle Band(LineRescueSim.TensionBand band)
    {
        switch (band)
        {
            case LineRescueSim.TensionBand.Hold:
                return new G12BandStyle { Hex = 0x39FF14, WidthMul = 1f, PulsePerSec = 0f };
            case LineRescueSim.TensionBand.NearSnap:
                // amber heat + widened breathing width: the creep tightening reads before the snap
                return new G12BandStyle { Hex = 0xFFB020, WidthMul = 1.35f, PulsePerSec = 3.2f };
            default:
                // live snap-tension segments vanish the same tick; the hot-red face feeds the T004 staging pass
                return new G12BandStyle { Hex = 0xFF3355, WidthMul = 1.2f, PulsePerSec = 0f };
```

```
            }
        }
        /// &lt;summary&gt;Worst live band across one segment range (one renderer run).&lt;/summary&gt;
        public static LineRescueSim.TensionBand WorstBandOfRun(IReadOnlyList&lt;LineRescueSim.Segment&gt; segs, int from
        {
            var worst = LineRescueSim.TensionBand.Hold;
            for (int i = from; i &lt;= to; i++)
            {
                var seg = segs[i];
                if (!seg.Live || seg.Load &lt;= 0f) continue;
                var band = LineRescueSim.Classify(seg.Load);
                if (band == LineRescueSim.TensionBand.Snap) return LineRescueSim.TensionBand.Snap;
                if (band == LineRescueSim.TensionBand.NearSnap) worst = LineRescueSim.TensionBand.NearSnap;
            }
            return worst;
        }
        /// &lt;summary&gt;Worst live band of a whole line.&lt;/summary&gt;
        public static LineRescueSim.TensionBand WorstBand(LineRescueSim.Line line)
        {
            return line.Segs.Count == 0 ? LineRescueSim.TensionBand.Hold : WorstBandOfRun(line.Segs, 0, line.Segs.Count
        }
    }
}
// G12InkEconomy.cs
using System;
namespace Sy.G12
{
    /// &lt;summary&gt;
    /// M5 ink economy core (task S1-T006): the per-level drawn-length budget and
    /// its settlement faces. Ink accounting charges the ACTUAL drawn span (one
    /// stroke clamps to MaxInk, mirroring the sim rule), so short strokes are
    /// cheap and the star rule "shorter draw = more stars" is an efficiency
    /// verdict, not a stroke counter. Settlement: stars come from the level's
    /// starInk thresholds (descending [1-star, 2-star, 3-star]), the one-stroke
    /// badge needs exactly one PLAYER draw (hint lines are paid ad seats and
    /// never enter this ledger), and a dry pen with the level unfinished is a
    /// level fail feeding the M4 revive staircase. The sweep statics (north-star
    /// tension window + window-center budget pick + star-threshold derivation)
    /// are pure so EditMode tests pin them without running the grid (G03
    /// TuneSweep paradigm). Pure C#, zero engine dependencies.
    /// &lt;/summary&gt;
    public sealed class G12InkEconomy
    {
        public const float MinStroke = 0.05f;                  // mirrors the sim tiny-span reject
        public const double WindowCenterUsedFraction = 0.725;   // tension window center
        public const double WindowMinUsedFraction = 0.55;       // looser = ink feels endless
        public const double WindowMaxUsedFraction = 0.90;       // tighter = wall-slam frustration
        public const float StarOneFactor = 1.25f;               // 25% sloppier than the skilled bot = 1 star
        public const float StarTwoFactor = 1.05f;               // within 5% of the skilled bot = 2 stars
        public const float StarThreeFactor = 0.85f;             // 15% shorter than the skilled bot = 3 stars
```

```
readonly float _budget;
float _used;
int _playerDraws;
public G12InkEconomy(float budget)
{
    _budget = budget &gt; 0f ? budget : LineRescueSim.MaxInk;
}
public float Budget { get { return _budget; } }
public float InkUsed { get { return _used; } }
public float InkLeft { get { return _budget - _used; } }
public int PlayerDrawCount { get { return _playerDraws; } }
public bool IsDry { get { return InkLeft &lt; MinStroke; } }
/// &lt;summary&gt;Ink meter fraction 1..0 (clamped) - the face the HUD reads.&lt;/summary&gt;
public float InkFraction()
{
    if (_budget &lt;= 0f) return 0f;
    float frac = InkLeft / _budget;
    return frac &lt; 0f ? 0f : (frac &gt; 1f ? 1f : frac);
}
/// &lt;summary&gt;
/// Charges one player stroke. The span clamps to MaxInk (the sim rule) and
/// the stroke is rejected when it does not fit the remaining ink - a
/// mid-stroke dry pen never draws a phantom line.
/// &lt;/summary&gt;
public bool TryConsumeDraw(float span)
{
    float s = span &gt; LineRescueSim.MaxInk ? LineRescueSim.MaxInk : span;
    if (s &lt; MinStroke) return false;
    if (s &gt; InkLeft + 1e-4f) return false;
    _used += s;
    _playerDraws++;
    return true;
}
// --- settlement faces ---
public sealed class Settlement
{
    public bool Cleared;
    public int Stars;
    public bool OneStroke;
    public float InkUsed;
    public int PlayerDraws;
    public bool DryFail;   // ink exhausted with the level unfinished (feeds M4)
}
/// &lt;summary&gt;Full settlement verdict for a finished level run.&lt;/summary&gt;
public static Settlement Settle(bool levelCleared, G12InkEconomy economy, float[] starInk)
{
    if (economy == null) throw new ArgumentNullException("economy");
    return new Settlement
    {
        Cleared = levelCleared,
```

```
        Stars = levelCleared ? StarsFor(economy.InkUsed, starInk) : 0,
        OneStroke = levelCleared &amp;&amp; OneStrokeBadge(economy.PlayerDrawCount),
        InkUsed = economy.InkUsed,
        PlayerDraws = economy.PlayerDrawCount,
        DryFail = DryLevelFail(economy.IsDry, levelCleared),
    };
}
/// &lt;summary&gt;
/// Star verdict from used ink against the descending thresholds
/// [1-star, 2-star, 3-star]: shorter draw = more stars; a clear above the
/// 1-star threshold still counts as a clear (0 stars - the efficiency
/// chase drives replays).
/// &lt;/summary&gt;
public static int StarsFor(float inkUsed, float[] starInk)
{
    if (starInk == null || starInk.Length != 3) return 0;
    if (inkUsed &lt;= starInk[2]) return 3;
    if (inkUsed &lt;= starInk[1]) return 2;
    if (inkUsed &lt;= starInk[0]) return 1;
    return 0;
}
/// &lt;summary&gt;One-stroke badge: exactly one PLAYER draw clears the level.&lt;/summary&gt;
public static bool OneStrokeBadge(int playerDraws)
{
    return playerDraws == 1;
}
/// &lt;summary&gt;
/// Dry-pen level fail: ink exhausted with the level unfinished - the M5
/// fail that feeds the M4 revive staircase.
/// &lt;/summary&gt;
public static bool DryLevelFail(bool isDry, bool levelCleared)
{
    return isDry &amp;&amp; !levelCleared;
}
// --- sweep statics (G03 TuneSweep paradigm: pure, test-pinned) ---
public struct SweepMetrics
{
    public bool Cleared;
    public float InkUsed;
    public int PlayerDraws;
    public int Fails;
    public int Saves;
}
public static double UsedFraction(SweepMetrics m, float budget)
{
    return budget &gt; 0f ? m.InkUsed / (double)budget : 0.0;
}
/// &lt;summary&gt;
/// North-star tension window: the skilled bot must clear inside the
/// candidate budget and its run must sit inside the used-fraction window
```

```
/// (no endless ink, no wall-slam).
/// &lt;/summary&gt;
public static bool Qualifies(SweepMetrics m, float budget)
{
    if (!m.Cleared || budget &lt;= 0f) return false;
    double f = UsedFraction(m, budget);
    return f &gt;= WindowMinUsedFraction &amp;&amp; f &lt;= WindowMaxUsedFraction;
}
/// &lt;summary&gt;
/// Window-center budget pick: among qualifying (budget, run) pairs choose
/// the one whose used fraction sits closest to the tension window center
/// (ties: the tighter budget). Entries are RE-CHECKED against the
/// qualification window (defense in depth: an unfiltered input can never
/// produce a silent pick). Returns null when nothing qualifies - the
/// honest fail path: thresholds are revisited with evidence, never
/// silently widened.
/// &lt;/summary&gt;
public static float? PickBudget(System.Collections.Generic.List&lt;System.Collections.Generic.KeyValuePair&lt;fl
{
    if (qualifying == null || qualifying.Count == 0) return null;
    float? best = null;
    double bestDev = 0.0;
    for (int i = 0; i &lt; qualifying.Count; i++)
    {
        float budget = qualifying[i].Key;
        var m = qualifying[i].Value;
        if (!Qualifies(m, budget)) continue;
        double dev = Math.Abs(UsedFraction(m, budget) - WindowCenterUsedFraction);
        if (best == null || dev &lt; bestDev || (dev == bestDev &amp;&amp; budget &lt; best.Value))
        {
            best = budget;
            bestDev = dev;
        }
    }
    return best;
}
/// &lt;summary&gt;
/// Star thresholds derived from the skilled bot's clearing ink: the bot
/// itself lands at 2 stars, beating it by 15% earns 3, being 25%
/// sloppier still earns 1. The 1-star threshold clamps below the budget
/// so the catalog validator bounds always hold.
/// &lt;/summary&gt;
public static float[] DeriveStarInk(float botInkUsed, float budget)
{
    if (botInkUsed &lt;= 0f || budget &lt;= 0f) throw new ArgumentException("bot ink and budget must be positive
    float t1 = botInkUsed * StarOneFactor;
    float t2 = botInkUsed * StarTwoFactor;
    float t3 = botInkUsed * StarThreeFactor;
    float cap = budget * 0.98f;
    if (t1 &gt; cap) t1 = cap;
```

```
            if (t2 &gt;= t1) t2 = t1 * 0.8f;
            if (t3 &gt;= t2) t3 = t2 * 0.8f;
            return new float[] { t1, t2, t3 };
        }
    }
}
// G12MainView.cs
using System.Collections.Generic;
using System.Threading.Tasks;
using Sy.Core;
using Sy.G12.Boot;
using Sy.Platform;
using UnityEngine;
using UnityEngine.EventSystems;
using UnityEngine.SceneManagement;
using UnityEngine.UI;
using Logger = Sy.Core.Logger;
namespace Sy.G12
{
    /// &lt;summary&gt;Minimal UGUI helpers for the G12 view (G06Ui paradigm, slice B subset).&lt;/summary&gt;
    internal static class G12Ui
    {
        public static Color Hex(int rgb)
        {
            return new Color(((rgb &gt;&gt; 16) &amp; 0xFF) / 255f, ((rgb &gt;&gt; 8) &amp; 0xFF) / 255f, (rgb &amp; 0xF
        }
        public static Text Label(RectTransform parent, string text, int size, Color color, TextAnchor anchor, bool bold)
        {
            var go = new GameObject("label", typeof(RectTransform), typeof(Text));
            var rect = (RectTransform)go.transform;
            rect.SetParent(parent, false);
            var t = go.GetComponent&lt;Text&gt;();
            t.text = string.IsNullOrEmpty(text) ? "?" : text;   // missing-key face (CoreCheck owns the fallback contrac
            t.fontSize = size;
            t.color = color;
            t.alignment = anchor;
            t.font = Resources.Load&lt;Font&gt;("Fonts/G12Main");
            if (bold) t.fontStyle = FontStyle.Bold;
            t.raycastTarget = false;
            return t;
        }
    }
    /// &lt;summary&gt;
    /// Scene wiring for the rescue board (task S1-T003 slices B+C): installs itself on
    /// the boot scene, configures the portrait board camera (night-black), renders
    /// player lines as neon LineRenderer runs (holes appear where segments snapped or
    /// got eaten; the line face escalates with the tension bands), renders falling
    /// hazards with the ArtRuntime fallback chain (procedural color blocks until the
    /// art table carries stems = U020 legal state), runs the single-finger draw input
    /// loop through the slice A presenter, shows the caption + first-draw guide beats
```

```
/// 1:1 from the sim event stream, and carries the slice C HUD: three hearts, the
/// ink meter and the band feedback visuals. The T004 staged performances land
/// with the view: the snap-collapse slow-mo + break flash, the eraser eat-line
/// threat face, and the revive offer seat (hybrid seat 1: the fail beat stages
/// a banner whose single tap restores play - fail counter reset, revive count
/// recorded - through the frozen sim).
/// The T005 hook face lands with the view: the cliff dog trembles while the
/// level plays, the lava ember rises through the T+0-3s hook window, and every
/// snap beat lands as a decaying camera punch (the first heavy hammer snapping
/// the first line reads as the first thrill - red line 11).
/// The T005 slice B sfx seat lands with the view: the signature hook's three
/// axes fire 1:1 on their beats (line_ready -&gt; sweep, tension -&gt; creak,
/// snap -&gt; crack), so the opening plays the "sweep -&gt; creak -&gt; snap" phrase
/// organically - the audio seat never throws in headless runs (G12Sfx).
    /// No scene asset edits: everything wires from code, so the headless PlayMode
    /// smoke can verify the whole chain (G06 MainView paradigm).
/// The S3-T004 slice A settle-ceremony layer lands with the view: the win
/// verdict plays as a beat sequence (victim rescue escape -&gt; heart salute -&gt;
/// star count-up -&gt; badge stamp) through the pure G12SettleFlow table, every
/// rescue flashes a relief pulse on the victim, every heart loss stages the
/// ember crack flicker + the near-miss quip, and the terminal fail adds the
/// focus scrim - all additive faces on top of the same-frame staging anchors
/// (settlement banner, revive offer), never gating a single one of them.
/// The S3-T004 slice B creep-focus layer lands with the view: a live run
/// entering the NearSnap band arms a held-breath close-up (an inset magnifier
/// panel on a render texture - the main board never moves), the frozen 0.55s
/// collapse slow-mo plays inside the inset when the line snaps, and the
/// inset retires with the strain when it eases (pure G12CreepFocus table).
/// &lt;/summary&gt;
public sealed class G12MainView : MonoBehaviour
{
    public const string SceneName = "S00_Boot";
    public const float FirstGuideAtSec = 0.05f;      // first-draw guide anchor (red line 11, E4 hook order)
    public const string FirstLevelId = "L01";
    public static readonly Color InkGreen = G12Ui.Hex(0x39FF14);   // neon marker green (GDD frozen)
    static readonly Color NightBg = G12Ui.Hex(0x05070A);
    static readonly Color TextMain = G12Ui.Hex(0xE8F6FF);
    static readonly Color TextDim = G12Ui.Hex(0x7FA0B8);
    static readonly Color RockColor = G12Ui.Hex(0xB8C4CC);
    static readonly Color GroundColor = G12Ui.Hex(0x18202C);
    static readonly Color VictimColor = G12Ui.Hex(0xFFC24A);
    const float LineWorldWidth = 0.055f;
    G12BoardPresenter _presenter;
    LineRescueSim _sim;
    RectTransform _canvasRect;
    RectTransform _safeRect;   // BR-U262-G12-2 chrome layer anchor (two-layer law)
    Text _caption;
    Text _guide;
    readonly List&lt;LineRenderer&gt; _linePool = new List&lt;LineRenderer&gt;();
    readonly List&lt;SpriteRenderer&gt; _hazardPool = new List&lt;SpriteRenderer&gt;();
```

```
Transform _world;
Material _lineMat;
Sprite _procSprite;
bool _guideFired;
bool _stroking;
Vector2 _strokeStart;
readonly List&lt;Image&gt; _hearts = new List&lt;Image&gt;();
Image _inkFill;
// T004 slice A staged performance state (snap collapse slow-mo + break flash)
float _slowmoLeft;
SpriteRenderer _snapFlash;
float _snapFlashLeft;
int _snapFlashCount;
// T004 slice B staged performance state (eraser eat-line toast + bite flash)
SpriteRenderer _eraseFlash;
float _eraseFlashLeft;
int _eraseFlashCount;
float _lastEatenWorldX;
Text _eraseToast;
float _eraseToastLeft;
Image _eraseToastIcon;   // S4-T003 slice 1b: eraser identity art rides the toast
// T004 slice C staged seat (revive offer banner; the whole banner is the accept control)
Button _reviveBanner;
int _reviveSeatAccepts;
// v1.1 (C1296): the revive seat degrades to a free-retry control when the
// ad path cannot deliver - the Failed phase must always own an exit
Text _reviveBannerLabel;
bool _reviveSeatFallback;
// T005 hook staging state (ten-second hook face: cliff tremble + lava approach + snap punch)
SpriteRenderer _victim;
Vector3 _victimBasePos;
SpriteRenderer _lavaGlow;
Vector3 _camBasePos;
float _snapShakeLeft;
int _snapShakeCount;
// T005 slice B sfx seat (signature hook axes staged 1:1 with their beats)
int _sfxSweepCount;
int _sfxCreakCount;
int _sfxSnapCount;
// T006 slice B settlement seat (cleared-level verdict: stars + badge + ink)
GameObject _settleRoot;
Text _settleTitle;
Text _settleStars;
Text _settleBadge;
Text _settleInk;
// S2-T007 slice B share seats: the badge brag button rides the settle
// banner (badge runs only - the card face never ships a lie), the
// help-me seat rides the fail face (the ask premise is a verified fail).
Button _settleShareBtn;
Button _helpSeatBtn;
```

```
        ["rescues_in_10s"] = t.RescuesInWindow,
    };
    return m;
}
// ---- verdicts: the truth as measured, GREEN within budget, RED outside,
//      never silenced for non-fast tiers (G06 honest-report paradigm) ----
static bool VHook(Dictionary&lt;string, float&gt; t)
{
    return t["caption_seen"] &gt;= 0f &amp;&amp; t["caption_seen"] &lt;= HookBudgetSec
        &amp;&amp; t["guide_seen"] &gt;= 0f &amp;&amp; t["guide_seen"] &lt;= GuideBudgetSec;
}
static bool VFirstOp(Dictionary&lt;string, float&gt; t)
{
    return t["first_own_op"] &gt;= 0f &amp;&amp; t["first_own_op"] &lt;= FirstOpBudgetSec;
}
static bool VFeedback(Dictionary&lt;string, float&gt; t)
{
    return t["first_line_lag"] &gt;= 0f &amp;&amp; t["first_line_lag"] &lt;= FeedbackLagBudgetSec;
}
static bool VSnap(Dictionary&lt;string, float&gt; t)
{
    return t["snap_lag"] &gt;= 0f &amp;&amp; t["snap_lag"] &lt;= SnapPerceiveBudgetSec;
}
static bool VWow(Dictionary&lt;string, float&gt; t)
{
    return t["wow_seen"] &gt;= 0f &amp;&amp; t["wow_seen"] &lt;= WowBudgetSec;
}
static bool VRescues(Dictionary&lt;string, float&gt; t)
{
    return t["rescues_in_10s"] &gt;= RescuesRequired;
}
static string VerdictLine(Pacing p, Dictionary&lt;string, float&gt; t)
{
    return "tier=" + p.Name
        + " hook&lt;=" + HookBudgetSec.ToString("0", CultureInfo.InvariantCulture) + "s: " + (VHook(t) ? "GREEN"
        + " | first_op&lt;=" + FirstOpBudgetSec.ToString("0", CultureInfo.InvariantCulture) + "s: " + (VFirstOp(
        + " | feedback_lag&lt;=" + FeedbackLagBudgetSec.ToString("0", CultureInfo.InvariantCulture) + "s: " + (V
        + " | snap&lt;=" + SnapPerceiveBudgetSec.ToString("0", CultureInfo.InvariantCulture) + "s_of_tension: "
        + " | wow&lt;=" + WowBudgetSec.ToString("0", CultureInfo.InvariantCulture) + "s: " + (VWow(t) ? "GREEN"
        + " | rescues&gt;=" + RescuesRequired + "in10s: " + (VRescues(t) ? "GREEN" : "RED");
}
static string Num(float v)
{
    return v &lt; 0f ? "null" : v.ToString("0.00", CultureInfo.InvariantCulture);
}
static string TierJson(Pacing p, Timeline t)
{
    var m = ModelTier(t);
    var sb = new StringBuilder();
    sb.Append("    { \"tier\": \"").Append(p.Name).Append("\",");
```

```
        sb.Append(" \"beats\": {");
        sb.Append(" \"caption\": ").Append(Num(m["caption_seen"])).Append(",");
        sb.Append(" \"guide\": ").Append(Num(m["guide_seen"])).Append(",");
        sb.Append(" \"first_own_op\": ").Append(Num(m["first_own_op"])).Append(",");
        sb.Append(" \"first_line_lag\": ").Append(Num(m["first_line_lag"])).Append(",");
        sb.Append(" \"first_tension\": ").Append(Num(m["first_tension"])).Append(",");
        sb.Append(" \"first_snap\": ").Append(Num(m["first_snap"])).Append(",");
        sb.Append(" \"snap_lag\": ").Append(Num(m["snap_lag"])).Append(",");
        sb.Append(" \"first_wow\": ").Append(Num(m["wow_seen"])).Append(",");
        sb.Append(" \"third_rescue\": ").Append(Num(m["third_rescue"])).Append(",");
        sb.Append(" \"rescues_in_10s\": ").Append(t.RescuesInWindow.ToString(CultureInfo.InvariantCulture)).Append("
        sb.Append(" \"fails_in_window\": ").Append(t.Fails.ToString(CultureInfo.InvariantCulture)).Append(",");
        sb.Append(" \"phase_failed\": ").Append(t.PhaseFailed ? "true" : "false");
        sb.Append(" }, \"verdicts\": {");
        sb.Append(" \"hook\": \"").Append(VHook(m) ? "GREEN" : "RED").Append("\",");
        sb.Append(" \"first_op\": \"").Append(VFirstOp(m) ? "GREEN" : "RED").Append("\",");
        sb.Append(" \"feedback\": \"").Append(VFeedback(m) ? "GREEN" : "RED").Append("\",");
        sb.Append(" \"snap\": \"").Append(VSnap(m) ? "GREEN" : "RED").Append("\",");
        sb.Append(" \"wow\": \"").Append(VWow(m) ? "GREEN" : "RED").Append("\",");
        sb.Append(" \"rescues\": \"").Append(VRescues(m) ? "GREEN" : "RED").Append("\"");
        sb.Append(" }, \"verdict_line\": \"").Append(VerdictLine(p, m)).Append("\" }");
        return sb.ToString();
    }
    // ---- fast tier: hard assertions (the only hard gate, U063 gate layering) ----
    [Test]
    public void FastPacing_HookGuideFirstOpAndFeedback_WithinWindows()
    {
        var t = ModelTier(CaptureTimeline(Fast));
        Assert.GreaterOrEqual(t["caption_seen"], 0f, "hook caption beat must exist");
        Assert.LessOrEqual(t["caption_seen"], HookBudgetSec,
            "hook frame + subtitle must land inside T+0-3s, modelled=" + t["caption_seen"]);
        Assert.LessOrEqual(t["guide_seen"], GuideBudgetSec,
            "the first-op guide must land inside 3s, modelled=" + t["guide_seen"]);
        Assert.LessOrEqual(t["first_own_op"], FirstOpBudgetSec,
            "the first own line must commit inside the 10s window, modelled=" + t["first_own_op"]);
        Assert.LessOrEqual(t["first_line_lag"], FeedbackLagBudgetSec,
            "first-line feedback must land &lt;=1s after the commit (same tick), measured=" + t["first_line_lag"]);
    }
    [Test]
    public void FastPacing_FirstSnapWow_WithinWindows()
    {
        var t = ModelTier(CaptureTimeline(Fast));
        Assert.GreaterOrEqual(t["snap_lag"], 0f,
            "the fast player's flick line must meet tension and snap inside the window");
        Assert.LessOrEqual(t["snap_lag"], SnapPerceiveBudgetSec,
            "the snap must be perceivable &lt;=3s after the first tension, measured=" + t["snap_lag"]);
        Assert.GreaterOrEqual(t["wow_seen"], 0f,
            "the first_wow banner must fire (the wave-zero heavy snaps the flick line)");
        Assert.LessOrEqual(t["wow_seen"], WowBudgetSec,
            "the first wow signal must land inside the 10s window, modelled=" + t["wow_seen"]);
```

```
}
[Test]
public void FastPacing_ThreeRescues_InsideTenSeconds()
{
    var t = ModelTier(CaptureTimeline(Fast));
    Assert.LessOrEqual(t["third_rescue"], FirstOpBudgetSec,
        "the third rescue must land inside the 10s window, modelled=" + t["third_rescue"]);
    Assert.GreaterOrEqual(t["rescues_in_10s"], RescuesRequired,
        "three rescues must land inside the 10s window, modelled=" + t["rescues_in_10s"]);
}
// ---- median / slow: honest verdicts recorded, never silenced ----
[Test]
public void MedianAndSlowPacing_Verdicts_Recorded_Honestly()
{
    foreach (var p in new[] { Median, Slow })
    {
        var t = ModelTier(CaptureTimeline(p));
        string verdict = VerdictLine(p, t);
        StringAssert.Contains(
            VHook(t) ? "hook&lt;=3s: GREEN" : "hook&lt;=3s: RED", verdict,
            p.Name + " hook verdict must state the measured truth");
        StringAssert.Contains(
            VFirstOp(t) ? "first_op&lt;=10s: GREEN" : "first_op&lt;=10s: RED", verdict,
            p.Name + " first_op verdict must state the measured truth");
        StringAssert.Contains(
            VWow(t) ? "wow&lt;=10s: GREEN" : "wow&lt;=10s: RED", verdict,
            p.Name + " wow verdict must state the measured truth");
        StringAssert.Contains(
            VRescues(t) ? "rescues&gt;=3in10s: GREEN" : "rescues&gt;=3in10s: RED", verdict,
            p.Name + " rescues verdict must state the measured truth");
    }
}
[Test]
public void ReportFile_CoversAllTiers_Timeline()
{
    var tiers = new List&lt;string&gt;();
    foreach (var p in new[] { Fast, Median, Slow })
    {
        var t = CaptureTimeline(p);
        tiers.Add(TierJson(p, t));
        // timeline sanity on the modelled clock: ops never precede their
        // decision beats, snaps never precede their first tension (a tier
        // that failed before committing a stroke reports -1 honestly).
        if (t.FirstOpSec &gt;= 0f)
        {
            Assert.GreaterOrEqual(t.FirstOpSec, p.HookRead + p.BoardScan,
                p.Name + " first own op must not precede the read+scan decision");
        }
        if (t.SnapSec &gt;= 0f &amp;&amp; t.TensionSec &gt;= 0f)
        {
```

```
                Assert.GreaterOrEqual(t.SnapSec, t.TensionSec,
                    p.Name + " snap must not precede the first tension");
            }
        }
        var sb = new StringBuilder();
        sb.Append("{\n");
        sb.Append("  \"gate\": \"g12-redline11\",\n");
        sb.Append("  \"generated\": \"").Append(DateTime.Now.ToString("yyyy-MM-dd HH:mm:ss", CultureInfo.InvariantCu
        sb.Append("  \"threshold_source\": \"docs/04 v1 (E4 H1-H5 re-cited verbatim; thresholds never silently widen
        sb.Append("  \"thresholds\": { \"hook_sec\": 3.0, \"guide_sec\": 3.0, \"first_op_sec\": 10.0, \"feedback_lag
        sb.Append("  \"tiers\": [\n");
        sb.Append(string.Join(",\n", tiers));
        sb.Append("\n  ]\n");
        sb.Append("}\n");
        string dir = Path.GetDirectoryName(ReportPath);
        if (!Directory.Exists(dir)) Directory.CreateDirectory(dir);
        File.WriteAllText(ReportPath, sb.ToString());
        Assert.IsTrue(File.Exists(ReportPath), "report file must exist");
        string text = File.ReadAllText(ReportPath);
        foreach (var tier in new[] { "\"tier\": \"fast\"", "\"tier\": \"median\"", "\"tier\": \"slow\"" })
        {
            StringAssert.Contains(tier, text, "report must contain " + tier);
        }
        StringAssert.Contains("\"thresholds\"", text, "report must carry the threshold block");
    }
    [Test]
    public void Flow_CommandKinds_BusinessOnly_NoTeaching_Login_OrAnnounce()
    {
        // Red line 11: no white-screen tutorial, no login wall, no announcement
        // popup - the event stream itself must stay a pure business flow.
        var t = CaptureTimeline(Fast);
        var businessKinds = new HashSet&lt;string&gt;
        {
            "caption", "guide_prompt", "line_ready", "line_tension", "line_snap",
            "first_wow", "rescue_success", "rescue_fail", "revive_offer",
            "hint_line_shown", "eraser_spawn", "eraser_eat"
        };
        foreach (var kind in t.Kinds)
        {
            Assert.IsTrue(businessKinds.Contains(kind),
                "event kind '" + kind + "' must be a business beat");
            foreach (var banned in new[] { "teach", "tutorial", "login", "announce", "popup", "notice" })
            {
                Assert.IsFalse(kind.ToLowerInvariant().Contains(banned),
                    "no " + banned + " beat may exist in the flow (kind=" + kind + ")");
            }
        }
    }
    [Test]
    public void FeedbackContract_CoreClipsUnderOneSecond_LeitmotifUnderHookLine()
```

```
{
    // First-core-op feedback latency contract: every core feedback clip
    // completes inside the 1s latency budget (same-tick wiring, G12Sfx
    // per beat). The leitmotif is the brand hook, not a core-op clip: its
    // own red line is the &lt;=3s hummable signature, asserted present here.
    string[] coreIds = { G12Sfx.Sweep, G12Sfx.Creak, G12Sfx.Snap };
    foreach (var id in coreIds)
    {
        float[] samples = G12Sfx.Synth(id);
        Assert.IsNotNull(samples, "clip " + id + " must synth");
        Assert.IsNotEmpty(samples, "clip " + id + " must have samples");
        float seconds = samples.Length / (float)G12Sfx.SampleRate;
        Assert.LessOrEqual(seconds, FeedbackLagBudgetSec,
            "clip " + id + " must complete within the 1s latency budget");
    }
    float[] hook = G12Sfx.Synth(G12Sfx.Leitmotif);
    Assert.IsNotEmpty(hook, "leitmotif must have samples");
    float hookSeconds = hook.Length / (float)G12Sfx.SampleRate;
    Assert.LessOrEqual(hookSeconds, 3f, "leitmotif must stay inside its 3s hook line");
}
// ---- S3-T003 slice b: the 60s shareable-moment window. Fast tier = the
//      only hard gate (U063 layering); median/slow verdicts land verbatim
//      in the machine-readable report Logs/share_report.json, never silenced.
static string ShareReportPath =&gt;
    Path.Combine(Application.dataPath, "..", "Logs", "share_report.json");
static bool VFirstShare(Timeline t)
{
    return t.ShareTimes.Count &gt; 0 &amp;&amp; t.ShareTimes[0] &lt;= FirstShareBudgetSec;
}
static bool VShareGap(Timeline t)
{
    return t.ShareTimes.Count &gt;= 2 &amp;&amp; t.ShareMaxGap &gt;= 0f &amp;&amp; t.ShareMaxGap &lt;= ShareWind
}
static bool VShareCount(Timeline t)
{
    return t.ShareTimes.Count &gt;= ShareFloorCount;
}
static string ShareVerdictLine(Pacing p, Timeline t)
{
    return "tier=" + p.Name
        + " share_first&lt;=" + FirstShareBudgetSec.ToString("0", CultureInfo.InvariantCulture) + "s: " + (VFirs
        + " | share_gap&lt;=" + ShareWindowSec.ToString("0", CultureInfo.InvariantCulture) + "s: " + (VShareGap(
        + " | share_count&gt;=" + ShareFloorCount + ": " + (VShareCount(t) ? "GREEN" : "RED");
}
static string ShareTierJson(Pacing p, Timeline t)
{
    var sb = new StringBuilder();
    sb.Append("    { \"tier\": \"").Append(p.Name).Append("\",");
    sb.Append(" \"beats\": {");
    sb.Append(" \"played_sec\": ").Append(Num(t.PlayedSec)).Append(",");
```

```
        sb.Append(" \"first_share\": ").Append(t.ShareTimes.Count &gt; 0 ? Num(t.ShareTimes[0]) : "null").Append(","
        sb.Append(" \"max_gap\": ").Append(Num(t.ShareMaxGap)).Append(",");
        sb.Append(" \"share_count\": ").Append(t.ShareTimes.Count.ToString(CultureInfo.InvariantCulture)).Append(","
        sb.Append(" \"fails\": ").Append(t.Fails.ToString(CultureInfo.InvariantCulture)).Append(",");
        sb.Append(" \"phase_failed\": ").Append(t.PhaseFailed ? "true" : "false");
        sb.Append(" }, \"verdicts\": {");
        sb.Append(" \"first_share\": \"").Append(VFirstShare(t) ? "GREEN" : "RED").Append("\",");
        sb.Append(" \"share_gap\": \"").Append(VShareGap(t) ? "GREEN" : "RED").Append("\",");
        sb.Append(" \"share_count\": \"").Append(VShareCount(t) ? "GREEN" : "RED").Append("\"");
        sb.Append(" }, \"verdict_line\": \"").Append(ShareVerdictLine(p, t)).Append("\" }");
        return sb.ToString();
    }
    [Test]
    public void FastPacing_ShareMoments_NoGapOverSixtySeconds()
    {
        var byTier = new Dictionary&lt;string, Timeline&gt;();
        foreach (var p in new[] { Fast, Median, Slow })
        {
            byTier[p.Name] = CaptureTimeline(p, ShareWindowSec);
        }
        var fast = byTier["fast"];
        // Hard gate (fast tier only): X032-1 - one shareable moment per &lt;=60s.
        Assert.IsNotEmpty(fast.ShareTimes,
            "the 60s sustained window must contain share-worthy beats");
        Assert.LessOrEqual(fast.ShareTimes[0], FirstShareBudgetSec,
            "the first share beat must ride the 10s wow promise, measured=" + Num(fast.ShareTimes[0]));
        Assert.GreaterOrEqual(fast.ShareTimes.Count, ShareFloorCount,
            "3 rescues + 1 wow are already promised inside 10s, measured=" + fast.ShareTimes.Count);
        Assert.GreaterOrEqual(fast.ShareMaxGap, 0f,
            "at least two share beats must land in the 60s window");
        Assert.LessOrEqual(fast.ShareMaxGap, ShareWindowSec,
            "no gap between share beats may exceed the 60s core-cycle cadence (X032-1), measured=" + Num(fast.ShareM
        // Median / slow: honest verdicts recorded verbatim, never silenced.
        var tierJsons = new List&lt;string&gt;();
        foreach (var p in new[] { Fast, Median, Slow })
        {
            var t = byTier[p.Name];
            tierJsons.Add(ShareTierJson(p, t));
            if (p == Fast) continue; // fast already hard-gated above
            string verdict = ShareVerdictLine(p, t);
            StringAssert.Contains(
                VFirstShare(t) ? "share_first&lt;=10s: GREEN" : "share_first&lt;=10s: RED", verdict,
                p.Name + " first-share verdict must state the measured truth");
            StringAssert.Contains(
                VShareGap(t) ? "share_gap&lt;=60s: GREEN" : "share_gap&lt;=60s: RED", verdict,
                p.Name + " share-gap verdict must state the measured truth");
            StringAssert.Contains(
                VShareCount(t) ? "share_count&gt;=4: GREEN" : "share_count&gt;=4: RED", verdict,
                p.Name + " share-count verdict must state the measured truth");
        }
```

```
            var sb = new StringBuilder();
            sb.Append("{\n");
            sb.Append("  \"gate\": \"g12-share-window\",\n");
            sb.Append("  \"generated\": \"").Append(DateTime.Now.ToString("yyyy-MM-dd HH:mm:ss", CultureInfo.InvariantCu
            sb.Append("  \"threshold_source\": \"docs/04 v2 S3-T003 slice b (X032-1: one shareable moment per &lt;=60s c
            sb.Append("  \"thresholds\": { \"window_sec\": 60.0, \"first_share_sec\": 10.0, \"max_gap_sec\": 60.0, \"sha
            sb.Append("  \"tiers\": [\n");
            sb.Append(string.Join(",\n", tierJsons));
            sb.Append("\n  ]\n");
            sb.Append("}\n");
            string dir = Path.GetDirectoryName(ShareReportPath);
            if (!Directory.Exists(dir)) Directory.CreateDirectory(dir);
            File.WriteAllText(ShareReportPath, sb.ToString());
            Assert.IsTrue(File.Exists(ShareReportPath), "share report file must exist");
            string text = File.ReadAllText(ShareReportPath);
            foreach (var tier in new[] { "\"tier\": \"fast\"", "\"tier\": \"median\"", "\"tier\": \"slow\"" })
            {
                StringAssert.Contains(tier, text, "share report must contain " + tier);
            }
            StringAssert.Contains("\"thresholds\"", text, "share report must carry the threshold block");
        }
    }
}
// G12SaveTests.cs
using System;
using NUnit.Framework;
using Sy.Core;
using UnityEngine;
namespace Sy.G12.Tests
{
    /// &lt;summary&gt;
    /// Save + level progression tests (task S2-T005, G06SaveTests paradigm):
    /// single-key JSON round-trip through the Sy.Core save service, the
    /// four-state lifecycle, corruption and future-version degradation, the
    /// version-migration skeleton, the ratchet semantics of RecordClear, the
    /// unlock-chain derivation (zero drift: unlock is computed, never stored)
    /// and the red line 12 zero-server audit (local PlayerPrefs is the only
    /// authority).
    /// &lt;/summary&gt;
    public class G12SaveTests
    {
        const string TestSlot = "g12_test_save";
        [TearDown]
        public void TearDown()
        {
            PlayerPrefs.DeleteKey(SaveService.Prefix + TestSlot);
        }
        static G12Profile SampleProfile()
        {
            var p = new G12Profile();
```

```
        G12Progress.RecordClear(p, "L01", 3, true);
        G12Progress.RecordClear(p, "L02", 2, false);
        G12Progress.RecordClear(p, "L05", 1, false);
        p.lastLevelId = "L02";
        return p;
    }
    [Test]
    public void RoundTrip_RestoresStarsBadgesAndLastLevel()
    {
        var save = new SaveService();
        G12SaveSystem.Persist(save, SampleProfile(), TestSlot);
        var loaded = new G12Profile();
        var status = G12SaveSystem.LoadInto(save, loaded, TestSlot);
        Assert.That(status, Is.EqualTo(G12LoadStatus.Loaded));
        Assert.That(loaded.levels.Count, Is.EqualTo(3));
        Assert.That(G12Progress.StarsOf(loaded, "L01"), Is.EqualTo(3));
        Assert.That(G12Progress.StarsOf(loaded, "L02"), Is.EqualTo(2));
        Assert.That(G12Progress.StarsOf(loaded, "L05"), Is.EqualTo(1));
        Assert.That(G12Progress.HasBadge(loaded, "L01"), Is.True);
        Assert.That(G12Progress.HasBadge(loaded, "L02"), Is.False);
        Assert.That(loaded.lastLevelId, Is.EqualTo("L02"));
    }
    [Test]
    public void EmptySlot_YieldsNewGameWithFreshProfile()
    {
        var save = new SaveService();
        var profile = SampleProfile(); // stale caller state must not survive a new game
        var status = G12SaveSystem.LoadInto(save, profile, TestSlot);
        Assert.That(status, Is.EqualTo(G12LoadStatus.NewGame));
        Assert.That(profile.levels.Count, Is.EqualTo(0));
        Assert.That(profile.lastLevelId, Is.EqualTo(""));
        Assert.That(G12Progress.StarsOf(profile, "L01"), Is.EqualTo(0));
    }
    [Test]
    public void CorruptedSlot_DegradesToFreshProfileWithoutThrowing()
    {
        PlayerPrefs.SetString(SaveService.Prefix + TestSlot, "{corrupted!!! not json");
        var save = new SaveService();
        var profile = SampleProfile();
        var status = G12SaveSystem.LoadInto(save, profile, TestSlot);
        Assert.That(status, Is.EqualTo(G12LoadStatus.CorruptedReset));
        Assert.That(profile.levels.Count, Is.EqualTo(0));
        Assert.That(profile.lastLevelId, Is.EqualTo(""));
    }
    [Test]
    public void FutureVersion_ResetsToFreshInsteadOfCrashing()
    {
        PlayerPrefs.SetString(SaveService.Prefix + TestSlot, "{\"g12v\":99,\"levels\":[],\"last\":\"L09\"}");
        var save = new SaveService();
        var profile = SampleProfile();
```

```
        var status = G12SaveSystem.LoadInto(save, profile, TestSlot);
        Assert.That(status, Is.EqualTo(G12LoadStatus.CorruptedReset));
        Assert.That(profile.levels.Count, Is.EqualTo(0));
        Assert.That(profile.lastLevelId, Is.EqualTo(""));
    }
    [Test]
    public void VersionlessLegacyPayload_MigratesWithDefaults()
    {
        PlayerPrefs.SetString(SaveService.Prefix + TestSlot, "{\"last\":\"L04\"}");
        var save = new SaveService();
        var profile = new G12Profile();
        var status = G12SaveSystem.LoadInto(save, profile, TestSlot);
        Assert.That(status, Is.EqualTo(G12LoadStatus.Migrated));
        Assert.That(profile.levels.Count, Is.EqualTo(0));       // missing fields fill defaults
        Assert.That(profile.lastLevelId, Is.EqualTo("L04"));    // known fields survive
    }
    [Test]
    public void RecordClear_RatchetsStarsAndBadgeOnlyUp()
    {
        var p = new G12Profile();
        Assert.That(G12Progress.RecordClear(p, "L01", 2, false), Is.True);
        Assert.That(G12Progress.RecordClear(p, "L01", 1, false), Is.False); // worse replay: no write
        Assert.That(G12Progress.StarsOf(p, "L01"), Is.EqualTo(2));
        Assert.That(G12Progress.RecordClear(p, "L01", 2, true), Is.True);   // badge lights up
        Assert.That(G12Progress.RecordClear(p, "L01", 3, false), Is.True); // stars still climb
        Assert.That(G12Progress.RecordClear(p, "L01", 0, false), Is.False); // zero-star replay: nothing
        Assert.That(G12Progress.StarsOf(p, "L01"), Is.EqualTo(3));
        Assert.That(G12Progress.HasBadge(p, "L01"), Is.True);               // badge never un-lights
        // clamped garbage never escapes the ratchet bounds
        Assert.That(G12Progress.RecordClear(p, "L02", 7, false), Is.True);
        Assert.That(G12Progress.StarsOf(p, "L02"), Is.EqualTo(G12SaveSystem.MaxStars));
        Assert.That(G12Progress.RecordClear(null, "L01", 3, true), Is.False); // null-safe
    }
    [Test]
    public void UnlockChain_FirstLevelOpenRestGatedOnPredecessorStars()
    {
        var catalog = G12GameCatalog.Load();
        Assert.That(catalog, Is.Not.Null, "cfg_g12_levels must load for chain tests");
        var p = new G12Profile();
        Assert.That(G12Progress.IsUnlocked(catalog, p, "L01"), Is.True);   // first level always open
        Assert.That(G12Progress.IsUnlocked(catalog, p, "L02"), Is.False);
        Assert.That(G12Progress.IsUnlocked(catalog, p, "L10"), Is.False);
        Assert.That(G12Progress.HighestUnlockedIndex(catalog, p), Is.EqualTo(0));
        G12Progress.RecordClear(p, "L01", 1, false);   // 1 star opens exactly one step
        Assert.That(G12Progress.IsUnlocked(catalog, p, "L02"), Is.True);
        Assert.That(G12Progress.IsUnlocked(catalog, p, "L03"), Is.False);
        G12Progress.RecordClear(p, "L02", 0, false);   // 0-star clear opens nothing
        Assert.That(G12Progress.IsUnlocked(catalog, p, "L03"), Is.False);
        Assert.That(G12Progress.ClearedCount(p), Is.EqualTo(1));
        G12Progress.RecordClear(p, "L02", 2, true);
```

```
    Assert.That(G12Progress.IsUnlocked(catalog, p, "L03"), Is.True);
    Assert.That(G12Progress.HighestUnlockedIndex(catalog, p), Is.EqualTo(2));
    Assert.That(G12Progress.ClearedCount(p), Is.EqualTo(2));
}
[Test]
public void RoundTripAcrossSave_UnlockDerivationZeroDrift()
{
    var catalog = G12GameCatalog.Load();
    Assert.That(catalog, Is.Not.Null, "cfg_g12_levels must load for chain tests");
    var save = new SaveService();
    var a = new G12Profile();
    G12Progress.RecordClear(a, "L01", 3, true);
    G12Progress.RecordClear(a, "L02", 2, false);
    G12SaveSystem.Persist(save, a, TestSlot);
    var b = new G12Profile();
    Assert.That(G12SaveSystem.LoadInto(save, b, TestSlot), Is.EqualTo(G12LoadStatus.Loaded));
    // the derivation verdicts must be identical on both sides of the save boundary
    for (int i = 0; i &lt; catalog.Levels.Length; i++)
    {
        string id = catalog.Levels[i].id;
        Assert.That(G12Progress.StarsOf(b, id), Is.EqualTo(G12Progress.StarsOf(a, id)), id);
        Assert.That(G12Progress.HasBadge(b, id), Is.EqualTo(G12Progress.HasBadge(a, id)), id);
        Assert.That(G12Progress.IsUnlocked(catalog, b, id), Is.EqualTo(G12Progress.IsUnlocked(catalog, a, id)),
    }
    Assert.That(G12Progress.HighestUnlockedIndex(catalog, b), Is.EqualTo(2));
}
[Test]
public void ServiceLevelChoke_DegradesToFreshWithoutThrowing()
{
    var save = new ThrowingSaveService();
    var profile = SampleProfile();
    Assert.DoesNotThrow(() =&gt;
    {
        var status = G12SaveSystem.LoadInto(save, profile, TestSlot);
        Assert.That(status, Is.EqualTo(G12LoadStatus.CorruptedReset));
    });
    Assert.That(profile.levels.Count, Is.EqualTo(0));
    Assert.That(profile.lastLevelId, Is.EqualTo(""));
}
[Test]
public void UnknownAndTamperedIds_StayLockedForever()
{
    var catalog = G12GameCatalog.Load();
    Assert.That(catalog, Is.Not.Null, "cfg_g12_levels must load for chain tests");
    var p = new G12Profile();
    // tampered rows can never unlock anything: derivation walks the catalog only
    Assert.That(G12Progress.RecordClear(p, "L99", 3, true), Is.True);
    Assert.That(G12Progress.RecordClear(p, "hack:extra", 3, true), Is.True);
    Assert.That(G12Progress.IsUnlocked(catalog, p, "L99"), Is.False);
    Assert.That(G12Progress.IsUnlocked(catalog, p, "L02"), Is.False);
```

```
            Assert.That(G12Progress.HighestUnlockedIndex(catalog, p), Is.EqualTo(0));
            // degenerate inputs are refused, not crashed
            Assert.That(G12Progress.IsUnlocked(null, p, "L01"), Is.False);
            Assert.That(G12Progress.IsUnlocked(catalog, null, "L01"), Is.False);
            Assert.That(G12Progress.IsUnlocked(catalog, p, ""), Is.False);
            Assert.That(G12Progress.RecordClear(p, "", 3, true), Is.False);
            Assert.That(G12Progress.HighestUnlockedIndex(null, p), Is.EqualTo(-1));
        }
        sealed class ThrowingSaveService : ISaveService
        {
            public void Save(string slot, string payloadJson) { throw new Exception("boom"); }
            public string Load(string slot) { throw new Exception("boom"); }
            public SyncResult CloudSync() { throw new Exception("boom"); }
        }
    }
}
// G12SettleFlowTests.cs
using NUnit.Framework;
using UnityEngine;
namespace Sy.G12.Tests
{
    /// &lt;summary&gt;
    /// EditMode pins for the S3-T004 slice A settle-ceremony flow table (pure
    /// math, no scene): the ceremony is the additive layer on top of the
    /// same-frame staging anchors, so these tests pin the phase order, the
    /// frame budget, the star count-up law, the badge window, the escape curve
    /// bounds, the crack flicker law, the scrim fade, the lost-slot mirror and
    /// the crack-face distinctness - the staged beats stay deterministic
    /// (G12StageStyle / G12HookFlow style-table sibling paradigm).
    /// &lt;/summary&gt;
    public sealed class G12SettleFlowTests
    {
        [Test]
        public void WinCeremonyFullVerdictReadsWithinFrameBudget()
        {
            Assert.IsTrue(G12SettleFlow.WinWithinBudget(),
                "the maximal verdict (3 stars + badge) reads inside the frame budget");
            Assert.LessOrEqual(G12SettleFlow.WinRestAtSec(3, true), G12SettleFlow.WinCeremonyBudgetSec);
            Assert.Greater(G12SettleFlow.WinRestAtSec(0, false), 0f,
                "even a zero-star run holds a verdict beat");
        }
        [Test]
        public void WinPhaseOrderWalksNoneRescueSaluteRevealBadgeRest()
        {
            Assert.AreEqual(G12SettleFlow.WinPhase.None, G12SettleFlow.PhaseAt(-0.01f, 3, true));
            Assert.AreEqual(G12SettleFlow.WinPhase.Rescue, G12SettleFlow.PhaseAt(0.10f, 3, true));
            Assert.AreEqual(G12SettleFlow.WinPhase.HeartSalute, G12SettleFlow.PhaseAt(0.50f, 3, true));
            Assert.AreEqual(G12SettleFlow.WinPhase.StarReveal, G12SettleFlow.PhaseAt(0.80f, 3, true));
            Assert.AreEqual(G12SettleFlow.WinPhase.StarReveal, G12SettleFlow.PhaseAt(1.40f, 3, true),
                "the third star pops at the window edge, reveal still owns the frame");
```

```
        Assert.AreEqual(G12SettleFlow.WinPhase.BadgeStamp, G12SettleFlow.PhaseAt(1.50f, 3, true));
        Assert.AreEqual(G12SettleFlow.WinPhase.Rest, G12SettleFlow.PhaseAt(1.60f, 3, true));
        Assert.AreEqual(G12SettleFlow.WinPhase.Rest, G12SettleFlow.PhaseAt(1.60f, 3, false),
            "a non-badge run rests right after the star window");
        Assert.AreEqual(G12SettleFlow.WinPhase.Rest, G12SettleFlow.PhaseAt(1.00f, 0, false),
            "a zero-star run rests after its one-step hold");
    }
    [Test]
    public void StarRevealCountsUpOnePerStepAndNeverExceedsEarned()
    {
        Assert.AreEqual(0, G12SettleFlow.StarRevealCountAt(0.69f, 2), "no star before the window opens");
        Assert.AreEqual(1, G12SettleFlow.StarRevealCountAt(0.70f, 2), "the first star pops at the window edge");
        Assert.AreEqual(1, G12SettleFlow.StarRevealCountAt(0.94f, 2));
        Assert.AreEqual(2, G12SettleFlow.StarRevealCountAt(0.95f, 2));
        Assert.AreEqual(2, G12SettleFlow.StarRevealCountAt(9f, 2), "clamped at the earned count");
        Assert.AreEqual(0, G12SettleFlow.StarRevealCountAt(9f, 0), "a zero-star run reveals nothing");
    }
    [Test]
    public void BadgeStampsAfterLastStarSlotOnlyForBadgeRuns()
    {
        float badgeAt = G12SettleFlow.BadgeStampAtSec(3);
        Assert.GreaterOrEqual(badgeAt, G12SettleFlow.StarRevealEndSec(3));
        Assert.AreEqual(badgeAt, G12SettleFlow.StarRevealEndSec(3) + G12SettleFlow.BadgeStampDelaySec);
        Assert.AreEqual(G12SettleFlow.WinPhase.StarReveal, G12SettleFlow.PhaseAt(1.40f, 3, true),
            "the last pop still reads as reveal");
        Assert.AreEqual(G12SettleFlow.WinPhase.BadgeStamp, G12SettleFlow.PhaseAt(badgeAt + 0.001f, 3, true));
        Assert.AreEqual(G12SettleFlow.WinPhase.Rest, G12SettleFlow.PhaseAt(badgeAt + 0.15f, 3, true));
        // a non-badge run never enters the stamp window at all
        Assert.AreNotEqual(G12SettleFlow.WinPhase.BadgeStamp, G12SettleFlow.PhaseAt(9f, 3, false));
    }
    [Test]
    public void EscapeCurveRisesEasedAndSettlesWithinBounds()
    {
        Assert.AreEqual(0f, G12SettleFlow.VictimEscapeY(0f), 1e-5f, "escape starts at the fail line");
        Assert.AreEqual(0f, G12SettleFlow.VictimEscapeY(-1f), 1e-5f, "negative time clamps to rest");
        float mid = G12SettleFlow.VictimEscapeY(G12SettleFlow.WinRescueSec * 0.5f);
        Assert.Greater(mid, 0f);
        Assert.Less(mid, G12SettleFlow.EscapeRiseWorld, "the rise eases in (mid-window below the top)");
        float top = G12SettleFlow.VictimEscapeY(G12SettleFlow.WinRescueSec);
        Assert.AreEqual(G12SettleFlow.EscapeRiseWorld, top, 1e-4f,
            "the rescue beat ends exactly at the safe height (continuous into the hops)");
        // deterministic + bounded through and past the bounce tail
        float a = G12SettleFlow.VictimEscapeY(G12SettleFlow.WinRescueSec + 0.12f);
        float b = G12SettleFlow.VictimEscapeY(G12SettleFlow.WinRescueSec + 0.12f);
        Assert.AreEqual(a, b, 0f, "same input, same offset (deterministic)");
        Assert.GreaterOrEqual(a, G12SettleFlow.EscapeRiseWorld - G12SettleFlow.EscapeBounceAmpWorld,
            "hops stay inside the amplitude");
        Assert.LessOrEqual(a, G12SettleFlow.EscapeRiseWorld, "never above the safe height");
        Assert.AreEqual(G12SettleFlow.EscapeRiseWorld, G12SettleFlow.VictimEscapeY(10f), 1e-5f,
            "settles at the top after the hops");
```

```
}
[Test]
public void SavePulseBounceIsADeterministicBell()
{
    Assert.AreEqual(0f, G12SettleFlow.SavePulseBounce(0f), 1e-5f);
    Assert.AreEqual(0f, G12SettleFlow.SavePulseBounce(1f), 1e-5f);
    Assert.AreEqual(1f, G12SettleFlow.SavePulseBounce(0.5f), 1e-4f, "peaks mid-window");
    Assert.AreEqual(G12SettleFlow.SavePulseBounce(0.3f), G12SettleFlow.SavePulseBounce(0.3f), 0f);
    Assert.LessOrEqual(G12SettleFlow.SavePulseBounce(0.8f), 1f);
    Assert.GreaterOrEqual(G12SettleFlow.SavePulseBounce(0.8f), 0f);
}
[Test]
public void LostSlotMirrorsTheLeftLitFillMap()
{
    // G12HudStyle fills slots 0..remaining-1; the slot that just went
    // dark is the new remaining index (rightmost drains first)
    Assert.AreEqual(2, G12SettleFlow.LostSlotFor(2));
    Assert.AreEqual(1, G12SettleFlow.LostSlotFor(1));
    Assert.AreEqual(0, G12SettleFlow.LostSlotFor(0));
    Assert.AreEqual(0, G12SettleFlow.LostSlotFor(-3), "out-of-range clamps to the leftmost slot");
    for (int remaining = 0; remaining &amp;lt; G12HudStyle.HeartSlots; remaining++)
    {
        int slot = G12SettleFlow.LostSlotFor(remaining);
        Assert.IsFalse(G12HudStyle.HeartFilled(slot, remaining),
            "the lost slot reads dim under the fill map (slot " + slot + ", remaining " + remaining + ")");
    }
}
[Test]
public void CrackPulseFlickersDeterministicallyAndTerminalBeatsHarder()
{
    Assert.AreEqual(0f, G12SettleFlow.CrackPulse(0f, false), 1e-5f, "window spent = flicker out");
    Assert.AreEqual(0f, G12SettleFlow.CrackPulse(0f, true), 1e-5f);
    float crack = G12SettleFlow.CrackPulse(0.5f, false);
    float terminal = G12SettleFlow.CrackPulse(0.5f, true);
    Assert.AreEqual(crack, G12SettleFlow.CrackPulse(0.5f, false), 0f, "deterministic");
    Assert.Greater(terminal, 0f, "the terminal crack reads alive mid-window");
    Assert.LessOrEqual(crack, 1f);
    Assert.LessOrEqual(terminal, 1f);
    // the terminal face carries more flicker inside the same window
    Assert.Greater(G12SettleFlow.TerminalCrackFlicksPerWindow, G12SettleFlow.CrackFlicksPerWindow,
        "the bottoming-out crack beats harder than a single loss");
}
[Test]
public void ScrimFadesToTargetAndHolds()
{
    Assert.AreEqual(0f, G12SettleFlow.ScrimAlphaAt(0f), 1e-5f);
    Assert.AreEqual(0f, G12SettleFlow.ScrimAlphaAt(-1f), 1e-5f);
    float half = G12SettleFlow.ScrimAlphaAt(G12SettleFlow.FailScrimFadeSec * 0.5f);
    Assert.Greater(half, 0f);
    Assert.Less(half, G12SettleFlow.FailScrimTargetAlpha, "mid-fade sits below the target");
```

```
            Assert.AreEqual(G12SettleFlow.FailScrimTargetAlpha,
                G12SettleFlow.ScrimAlphaAt(G12SettleFlow.FailScrimFadeSec), 1e-5f,
                "target reached at the fade edge");
            Assert.AreEqual(G12SettleFlow.FailScrimTargetAlpha,
                G12SettleFlow.ScrimAlphaAt(10f), 1e-5f, "holds after the fade");
        }
        [Test]
        public void HeartCrackFaceStandsApartFromEveryStagedFace()
        {
            Assert.IsTrue(G12SettleFlow.HeartCrackFaceDistinct(),
                "the crack ember is its own face - never a collapse, bite, offer, hint, lava, heart or band");
        }
        [Test]
        public void BeatWindowsStaySnappy()
        {
            Assert.LessOrEqual(G12SettleFlow.WinRestAtSec(3, true), G12SettleFlow.WinCeremonyBudgetSec,
                "the full verdict reads inside the frame budget");
            Assert.LessOrEqual(G12SettleFlow.FailBeatWindowSec, 0.5f, "a heart crack reads fast, never a stall");
            Assert.LessOrEqual(G12SettleFlow.SavePulseWindowSec, 0.5f, "the rescue flash reads fast");
            Assert.LessOrEqual(G12SettleFlow.FailScrimFadeSec, G12SettleFlow.FailBeatWindowSec,
                "the scrim settles inside the fail beat window");
        }
    }
}
// G12SfxTests.cs
using System;
using NUnit.Framework;
using Sy.G12;
namespace Sy.G12.Tests
{
    /// &lt;summary&gt;
    /// Determinism + X028 discipline tests for the procedural sfx (task S1-T005
    /// slice B): synthesis is pure math (no Random, no files) so every render is
    /// identical; the signature hook fits the 3s hummable window and is the exact
    /// composition of its three axes; the strain axis stays the dark voice
    /// against the two noisy faces; the creak tightens through an accelerating
    /// stick-slip pulse train; the snap reads as a crack (bright burst settling
    /// dark), never as a low solid knock.
    /// &lt;/summary&gt;
    public class G12SfxTests
    {
        static int ZeroCrossings(float[] buf, int from, int to)
        {
            int n = 0;
            int end = Math.Min(to, buf.Length);
            for (int i = Math.Max(1, from); i &lt; end; i++)
                if ((buf[i - 1] &lt;= 0f &amp;&amp; buf[i] &gt; 0f) || (buf[i - 1] &gt;= 0f &amp;&amp; buf[i] &lt; 0f))
            return n;
        }
        static double CrossingRate(float[] buf, double sec)
```

```
{
    return ZeroCrossings(buf, 0, buf.Length) / sec;
}
static double Energy(float[] buf, int from, int to)
{
    double e = 0.0;
    int end = Math.Min(to, buf.Length);
    for (int i = Math.Max(0, from); i &lt; end; i++) e += (double)buf[i] * buf[i];
    return e;
}
[Test]
public void Synth_Is_Deterministic_And_Bounded_For_Every_Clip()
{
    string[] ids = { G12Sfx.Sweep, G12Sfx.Creak, G12Sfx.Snap, G12Sfx.Leitmotif };
    foreach (var id in ids)
    {
        var a = G12Sfx.Synth(id);
        var b = G12Sfx.Synth(id);
        Assert.Greater(a.Length, 0, "clip must synthesize: " + id);
        Assert.AreEqual(a.Length, b.Length, "the same recipe keeps one length: " + id);
        for (int i = 0; i &lt; a.Length; i++)
        {
            Assert.AreEqual(a[i], b[i], "synthesis must be deterministic (no Random): " + id);
            Assert.IsFalse(float.IsNaN(a[i]) || float.IsInfinity(a[i]), "samples must be finite: " + id);
            Assert.LessOrEqual(Math.Abs(a[i]), 1f, "normalized samples stay inside [-1,1]: " + id);
            Assert.LessOrEqual(Math.Abs(a[i]), (float)G12Sfx.SamplePeakCap + 1e-4f,
                "TP -3 dB caliber: the loudness family caps sample peak (S3-T002): " + id);
        }
    }
    // unknown ids fold to silence, never throw
    Assert.AreEqual(0, G12Sfx.Synth("no_such_clip").Length, "unknown clip ids synthesize empty");
}
[Test]
public void Signature_Hook_Is_The_Exact_Composition_Of_Its_Three_Axes()
{
    var hook = G12Sfx.Synth(G12Sfx.Leitmotif);
    double sec = hook.Length / (double)G12Sfx.SampleRate;
    Assert.LessOrEqual(sec, 3.0, "X028: the hook must be hummable within 3s");
    Assert.GreaterOrEqual(sec, 0.30, "the hook must be a real phrase, not a blip");
    // the signature is literally sweep + gap + creak + gap + snap: the
    // exact composition is the assertion oracle (deterministic buffers)
    var sweep = G12Sfx.Synth(G12Sfx.Sweep);
    var creak = G12Sfx.Synth(G12Sfx.Creak);
    var snap = G12Sfx.Synth(G12Sfx.Snap);
    int gap1 = (int)(G12Sfx.SampleRate * G12Sfx.LeitmotifGap1Sec);
    int gap2 = (int)(G12Sfx.SampleRate * G12Sfx.LeitmotifGap2Sec);
    Assert.AreEqual(sweep.Length + gap1 + creak.Length + gap2 + snap.Length, hook.Length,
        "the leitmotif must contain all three axes in order (sweep -&gt; creak -&gt; snap)");
    // segment-wise equality (S3-T002): the phrase is the exact calibrated
    // acts the game plays, sample for sample - no phrase-level rescale
```

```
    int creakAt = sweep.Length + gap1;
    int snapAt = creakAt + creak.Length + gap2;
    for (int i = 0; i &lt; sweep.Length; i++)
        Assert.AreEqual(sweep[i], hook[i], "hook sweep segment must equal the calibrated sweep act at " + i);
    for (int i = 0; i &lt; creak.Length; i++)
        Assert.AreEqual(creak[i], hook[creakAt + i], "hook creak segment must equal the calibrated creak act at
    for (int i = 0; i &lt; snap.Length; i++)
        Assert.AreEqual(snap[i], hook[snapAt + i], "hook snap segment must equal the calibrated snap act at " +
}
[Test]
public void Loudness_Family_Matches_Loudnorm_Caliber_Spread_And_Meter_Scale()
{
    // S3-T002 code leg: the ffmpeg-loudnorm family caliber - the three
    // acts are K-weighted-equal (same-batch loudness difference &lt;= 1 dB)
    // at the TP-legal ceiling; the hook phrase rides on the acts by
    // construction, so its measured value is recorded, not gated.
    string[] acts = { G12Sfx.Sweep, G12Sfx.Creak, G12Sfx.Snap };
    double min = double.MaxValue, max = double.MinValue;
    foreach (var id in acts)
    {
        var b = G12Sfx.Synth(id);
        double lufs = G12Sfx.MeasureLoudnessLufs(b);
        min = Math.Min(min, lufs);
        max = Math.Max(max, lufs);
    }
    Assert.LessOrEqual(max - min, G12Sfx.FamilySpreadMaxDb,
        "section-2 acceptance: same-batch loudness difference must stay within 1 dB (spread="
        + (max - min).ToString("F3") + " dB)");
    // the meter itself keeps the BS.1770 scale: a -20 dBFS 1 kHz sine is
    // the EBU calibration face and must read about -23 LUFS (K-weighting
    // at 1 kHz + the -0.691 offset cancel to the R128 reference)
    var tone = new float[G12Sfx.SampleRate];
    for (int i = 0; i &lt; tone.Length; i++)
        tone[i] = (float)(Math.Sin(2.0 * Math.PI * 1000.0 * i / G12Sfx.SampleRate) * Math.Pow(10.0, -20.0 / 20.0
    Assert.AreEqual(-23.0, G12Sfx.MeasureLoudnessLufs(tone), 0.35,
        "K-weighted meter keeps the BS.1770 scale (1 kHz sine at -20 dBFS reads about -23 LUFS)");
}
[Test]
public void Three_Axes_Stay_Spectrally_Distinct_Voices()
{
    // X028 mutual-exclusion contour check: the strain axis is the dark
    // voice of the phrase; the swipe and the crack stay the noisy faces.
    var sweep = G12Sfx.Synth(G12Sfx.Sweep);
    var creak = G12Sfx.Synth(G12Sfx.Creak);
    var snap = G12Sfx.Synth(G12Sfx.Snap);
    double sweepRate = CrossingRate(sweep, sweep.Length / (double)G12Sfx.SampleRate);
    double creakRate = CrossingRate(creak, creak.Length / (double)G12Sfx.SampleRate);
    Assert.Greater(sweepRate, creakRate * 2.0,
        "the swipe (noisy friction) must cross zero far faster than the strain (dark creak)");
    // the crack's face is its burst window (the release tail settling dark
```

```
        // is its own check below): the burst must stay brighter than the
        // whole strain bed
        double snapSec = snap.Length / (double)G12Sfx.SampleRate;
        double burstRate = ZeroCrossings(snap, 0, snap.Length / 5) / (snapSec * 0.2);
        Assert.Greater(burstRate, creakRate * 2.0,
            "the crack burst must stay brighter than the strain bed");
        // the swipe keeps its own shape: front-loaded friction, not a swell
        int n = sweep.Length;
        double front = Energy(sweep, 0, (int)(n * 0.30));
        double tail = Energy(sweep, (int)(n * 0.70), n);
        Assert.Greater(front, tail * 4.0,
            "the swipe must read front-loaded (ink pushes, then lifts)");
    }
    [Test]
    public void Creak_Tightens_Through_Accelerating_Pulses()
    {
        var creak = G12Sfx.Synth(G12Sfx.Creak);
        int n = creak.Length;
        // the strain brightens as it tightens: the pulse pitches climb
        int q1 = ZeroCrossings(creak, 0, n / 4);
        int q4 = ZeroCrossings(creak, 3 * n / 4, n);
        Assert.Greater(q4, q1 * 1.5,
            "the creak must tighten (brighter crossings late than early)");
        // stick-slip train: the coarse 1ms envelope must erupt into several
        // distinct pulse groups above 0.60 x peak, re-armed below 0.40 x peak
        // (the drone bed sits near 0.33 x peak, so the re-arm splits pulses)
        double peak = 0.0;
        int win = Math.Max(1, G12Sfx.SampleRate / 1000);
        for (int i = 0; i &lt; n; i++) peak = Math.Max(peak, Math.Abs(creak[i]));
        int groups = 0;
        bool armed = true;
        for (int i = 0; i &lt; n; i += win)
        {
            double e = 0.0;
            int end = Math.Min(n, i + win);
            for (int k = i; k &lt; end; k++) e = Math.Max(e, Math.Abs(creak[k]));
            if (armed &amp;&amp; e &gt; 0.60 * peak) { groups++; armed = false; }
            else if (!armed &amp;&amp; e &lt; 0.40 * peak) armed = true;
        }
        Assert.GreaterOrEqual(groups, 5,
            "the strain must read as multiple stick-slip pulses, not one moan (found " + groups + ")");
    }
    [Test]
    public void Snap_Reads_As_A_Crack_Not_A_Knock()
    {
        var snap = G12Sfx.Synth(G12Sfx.Snap);
        int n = snap.Length;
        double sec = n / (double)G12Sfx.SampleRate;
        // the crack is the attack: the burst must reach near-peak inside 3ms
        double peak = 0.0;
```

```
            for (int i = 0; i &lt; n; i++) peak = Math.Max(peak, Math.Abs(snap[i]));
            double attack = 0.0;
            int aEnd = Math.Min(n, (int)(G12Sfx.SampleRate * 0.003));
            for (int i = 0; i &lt; aEnd; i++) attack = Math.Max(attack, Math.Abs(snap[i]));
            Assert.GreaterOrEqual(attack, 0.75 * peak,
                "the snap must hit hard within its first 3ms (a crack, not a swell)");
            // the contour settles dark: the burst opens bright, the release thump
            // closes low - the inverse weight of a low solid knock
            double openRate = ZeroCrossings(snap, 0, n / 5) / (sec * 0.2);
            double closeRate = ZeroCrossings(snap, n / 2, n) / (sec * 0.5);
            Assert.Greater(openRate, closeRate * 2.0,
                "the snap must open bright and settle dark (crack face, never a knock)");
        }
    }
}
// G12StageStyleTests.cs
using NUnit.Framework;
namespace Sy.G12.Tests
{
    /// &lt;summary&gt;
    /// Task S1-T004 staging pins: slice A (snap-collapse budget window 0.3-0.8s at
    /// quarter speed, the C144-reserved hot-red break flash), slice B (eraser
    /// eat-line face: bite window pinned to the frozen sim cadence, threat pink
    /// distinct from every band face, bite lighter than collapse) and slice C
    /// (revive offer seat: victim-gold banner face distinct from every staged
    /// threat, geometry fits the portrait canvas). Pure constants, no scene -
    /// EditMode pins the whole table (G12HudStyleTests sibling).
    /// &lt;/summary&gt;
    public sealed class G12StageStyleTests
    {
        [Test]
        public void SlowmoBudgetWindowMatchesTaskBook()
        {
            Assert.GreaterOrEqual(G12StageStyle.SlowmoBudgetLoSec, 0.3f, "budget floor = task book 0.3s");
            Assert.LessOrEqual(G12StageStyle.SlowmoBudgetHiSec, 0.8f, "budget ceiling = task book 0.8s");
            Assert.LessOrEqual(G12StageStyle.SlowmoBudgetLoSec, G12StageStyle.SlowmoBudgetHiSec,
                "window must be well-formed (floor never above ceiling)");
        }
        [Test]
        public void StagedDurationSitsInsideBudgetWindow()
        {
            Assert.IsTrue(G12StageStyle.SlowmoWithinBudget(),
                "staged duration must sit inside the 0.3-0.8s task-book budget");
        }
        [Test]
        public void SlowmoScaleSlowsButNeverStopsTime()
        {
            Assert.Greater(G12StageStyle.SlowmoScale, 0f, "scale must keep sim time flowing");
            Assert.Less(G12StageStyle.SlowmoScale, 1f, "scale must actually slow the collapse beat");
        }
```

```
    [Test]
    public void SnapFlashCarriesTheReservedHotRed()
    {
        Assert.AreEqual(0xFF3355, G12StageStyle.SnapFlashHex,
            "C144 reserved the Snap band hot-red for the T004 stage face");
        Assert.Greater(G12StageStyle.SnapFlashWorldSize, 0f, "flash must be visible on the board");
    }
    [Test]
    public void EraseBiteWindowPinnedToSimCadence()
    {
        Assert.AreEqual(0.5f, G12StageStyle.EraseBiteWindowSec,
            "task book pins the eraser bite cadence at 0.5s (x3 per wave)");
        Assert.AreEqual(LineRescueSim.EraserEatInterval, G12StageStyle.EraseBiteWindowSec,
            "staged bite window must equal the frozen sim bite cadence");
        Assert.IsTrue(G12StageStyle.EraseBiteWindowMatchesSim(),
            "pure helper must mirror the frozen cadence (drift guard)");
    }
    [Test]
    public void EraseFlashCarriesItsOwnThreatPink()
    {
        Assert.AreEqual(0xFF9FB2, G12StageStyle.EraseFlashHex, "eraser threat pins its own pink");
        Assert.AreNotEqual(G12StageStyle.SnapFlashHex, G12StageStyle.EraseFlashHex,
            "a bite must never read as a collapse (distinct from the snap hot-red)");
        Assert.AreNotEqual(G12HudStyle.Band(LineRescueSim.TensionBand.Hold).Hex, G12StageStyle.EraseFlashHex,
            "distinct from the Hold neon green");
        Assert.AreNotEqual(G12HudStyle.Band(LineRescueSim.TensionBand.NearSnap).Hex, G12StageStyle.EraseFlashHex,
            "distinct from the NearSnap amber");
        Assert.AreNotEqual(G12HudStyle.Band(LineRescueSim.TensionBand.Snap).Hex, G12StageStyle.EraseFlashHex,
            "distinct from the Snap band red");
    }
    [Test]
    public void EraseFlashVisibleButLighterThanCollapse()
    {
        Assert.Greater(G12StageStyle.EraseFlashWorldSize, 0f, "bite flash must be visible on the board");
        Assert.Less(G12StageStyle.EraseFlashWorldSize, G12StageStyle.SnapFlashWorldSize,
            "one bite reads lighter than a full collapse (visual hierarchy)");
    }
    [Test]
    public void ReviveOfferFaceDistinctFromEveryStagedFace()
    {
        Assert.IsTrue(G12StageStyle.ReviveOfferFaceDistinct(),
            "a second chance must never read as a collapse, a bite, or a band face");
    }
    [Test]
    public void ReviveOfferCarriesTheVictimGold()
    {
        Assert.AreEqual(0xFFC24A, G12StageStyle.ReviveOfferHex,
            "the offer speaks for the stranded victim (victim gold identity)");
        Assert.Greater(G12StageStyle.ReviveOfferTextSize, 0f, "offer text must render");
    }
```

```
        [Test]
        public void ReviveOfferBannerGeometrySane()
        {
            Assert.Greater(G12StageStyle.ReviveOfferBannerW, 0f, "banner must exist");
            Assert.Greater(G12StageStyle.ReviveOfferBannerH, 0f, "banner must exist");
            Assert.LessOrEqual(G12StageStyle.ReviveOfferBannerW, G12BoardLayout.RefW,
                "banner fits the portrait canvas width");
            Assert.LessOrEqual(G12StageStyle.ReviveOfferAnchorY, G12BoardLayout.RefH * 0.5f,
                "banner anchor stays inside the canvas (top)");
            Assert.GreaterOrEqual(G12StageStyle.ReviveOfferAnchorY, -G12BoardLayout.RefH * 0.5f,
                "banner anchor stays inside the canvas (bottom)");
        }
    }
}
// G12TelemetryTests.cs
using System.Collections.Generic;
using NUnit.Framework;
using Sy.G12.Boot;
using Sy.Telemetry;
using UnityEngine;
namespace Sy.G12.Tests
{
    /// &lt;summary&gt;
    /// evt event-family specialized suite (task S2-T006, G12AdCapTests
    /// paradigm): the six section-8 seats end to end - the wire loads the
    /// section schema, the shared enum pins the exact seven ids (test-side
    /// oracle = a rename anywhere breaks here first), the level funnel orders
    /// attempts honestly (fresh level = 1, same-level restart bumps, a rescue
    /// continues the current run), the snap/eat beats carry the band and the
    /// located anchor, the evt_ad payload gains its design-table category, the
    /// evt_share dock lands card_type, and the schema's required-field
    /// enforcement is live (a missing attempts field is named, never silent).
    /// &lt;/summary&gt;
    public sealed class G12TelemetryTests
    {
        [Test]
        public void Wire_Service_Loads_The_Section_Schema()
        {
            var telemetry = G12TelemetryWire.CreateService();
            Assert.AreEqual(7, telemetry.SchemaEventCount,
                "the wire must load all seven section-8 events from cfg_g12_events");
        }
        [Test]
        public void Shared_Enum_Pins_The_Section_Ids()
        {
            Assert.AreEqual("evt_level_start", EventIds.LevelStart);
            Assert.AreEqual("evt_level_complete", EventIds.LevelComplete);
            Assert.AreEqual("evt_level_fail", EventIds.LevelFail);
            Assert.AreEqual("evt_line_snap", EventIds.LineSnap);
            Assert.AreEqual("evt_eraser_eat", EventIds.EraserEat);
```

```
    Assert.AreEqual("evt_ad", EventIds.Ad);
    Assert.AreEqual("evt_share", EventIds.Share);
}
[Test]
public void Start_Fires_First_Attempt_With_Zero_Ink()
{
    var sink = new CaptureSink();
    var events = new G12TelemetryEvents(Service(sink));
    events.NotifyLevelStart("L01");
    Assert.AreEqual(1, sink.Ids.Count);
    Assert.AreEqual(EventIds.LevelStart, sink.Ids[0]);
    Assert.AreEqual("L01", sink.Fields[0]["level_id"]);
    Assert.AreEqual("0", sink.Fields[0]["ink_used"]);
    Assert.AreEqual("1", sink.Fields[0]["attempts"]);
    Assert.AreEqual(1, events.CurrentAttempt);
}
[Test]
public void Fail_Reports_Current_Attempt_And_Spent_Ink()
{
    var sink = new CaptureSink();
    var events = new G12TelemetryEvents(Service(sink));
    events.NotifyLevelStart("L01");
    events.NotifyLevelFail("L01", 2.4f);
    Assert.AreEqual(EventIds.LevelFail, sink.Ids[1]);
    Assert.AreEqual("L01", sink.Fields[1]["level_id"]);
    Assert.AreEqual("2.4", sink.Fields[1]["ink_used"]);
    Assert.AreEqual("1", sink.Fields[1]["attempts"],
        "the failed run reports its own ordinal - the rescue continues it");
}
[Test]
public void Complete_Reports_Settlement_Ink_On_The_Rescued_Attempt()
{
    var sink = new CaptureSink();
    var events = new G12TelemetryEvents(Service(sink));
    events.NotifyLevelStart("L01");
    events.NotifyLevelFail("L01", 1.8f);
    events.NotifyLevelComplete("L01", 3.6f);
    Assert.AreEqual(EventIds.LevelComplete, sink.Ids[2]);
    Assert.AreEqual("3.6", sink.Fields[2]["ink_used"]);
    Assert.AreEqual("1", sink.Fields[2]["attempts"],
        "a revived clear reports the attempt the near-miss closed");
}
[Test]
public void Same_Level_Restart_Bumps_And_Level_Switch_Resets()
{
    var sink = new CaptureSink();
    var events = new G12TelemetryEvents(Service(sink));
    events.NotifyLevelStart("L01");
    events.NotifyLevelStart("L01");
    events.NotifyLevelStart("L02");
```

```
    Assert.AreEqual("1", sink.Fields[0]["attempts"]);
    Assert.AreEqual("2", sink.Fields[1]["attempts"],
        "restarting the same level is a new run ordinal");
    Assert.AreEqual("1", sink.Fields[2]["attempts"],
        "switching levels resets the ordinal");
    Assert.AreEqual("L02", events.CurrentLevelId);
}
[Test]
public void Snap_And_Eat_Carry_Band_And_Located_Anchor()
{
    var sink = new CaptureSink();
    var events = new G12TelemetryEvents(Service(sink));
    events.NotifyLineSnap("L04", "Snap");
    events.NotifyEraserEat("L04", "Top@1.25");
    Assert.AreEqual(EventIds.LineSnap, sink.Ids[0]);
    Assert.AreEqual("L04", sink.Fields[0]["level_id"]);
    Assert.AreEqual("Snap", sink.Fields[0]["tension_band"]);
    Assert.AreEqual(EventIds.EraserEat, sink.Ids[1]);
    Assert.AreEqual("Top@1.25", sink.Fields[1]["anchor_id"]);
}
[Test]
public void Share_Dock_Lands_Card_Type()
{
    var sink = new CaptureSink();
    var events = new G12TelemetryEvents(Service(sink));
    events.NotifyShare("L02", "onestroke");
    Assert.AreEqual(1, sink.Ids.Count);
    Assert.AreEqual(EventIds.Share, sink.Ids[0]);
    Assert.AreEqual("L02", sink.Fields[0]["level_id"]);
    Assert.AreEqual("onestroke", sink.Fields[0]["card_type"]);
}
[Test]
public void Ad_Funnel_Payload_Carries_The_Design_Category()
{
    var sink = new CaptureSink();
    var funnel = new G12AdFunnel(Service(sink), null, null);
    funnel.ReportOffer(G12AdPoints.Revive, "probe");
    funnel.ReportBlocked(G12AdPoints.Hint, "daily_cap");
    funnel.ReportOffer(G12AdPoints.InkDouble, "probe");
    Assert.AreEqual("rescue", sink.Fields[0]["category"]);
    Assert.AreEqual("direct", sink.Fields[1]["category"]);
    Assert.AreEqual("double", sink.Fields[2]["category"]);
    Assert.AreEqual("offer", sink.Fields[0]["state"],
        "the five-state audit payload is unchanged next to the new field");
}
[Test]
public void Schema_Enforces_Required_Fields_Live()
{
    var telemetry = new TelemetryService();
    var asset = Resources.Load&lt;TextAsset&gt;("cfg_g12_events");
```

```
            Assert.NotNull(asset, "cfg_g12_events loadable from Resources");
            Assert.IsTrue(telemetry.LoadSchemaFromJson(asset.text));
            var sink = new CaptureSink();
            telemetry.AddSink(sink);
            telemetry.Track(EventIds.LevelStart, new Dictionary&lt;string, string&gt; { { "level_id", "L01" }, { "ink_us
            Assert.AreEqual(1, telemetry.LastWarnings.Count, "a missing attempts field is named, never silent");
            StringAssert.Contains("attempts", telemetry.LastWarnings[0]);
            Assert.AreEqual("attempts", sink.Fields[0]["__missing"],
                "the sink payload carries the machine-readable missing marker");
        }
        [Test]
        public void Valid_Section_Payloads_Produce_Zero_Warnings()
        {
            var telemetry = new TelemetryService();
            var asset = Resources.Load&lt;TextAsset&gt;("cfg_g12_events");
            Assert.NotNull(asset, "cfg_g12_events loadable from Resources");
            Assert.IsTrue(telemetry.LoadSchemaFromJson(asset.text));
            // Test-side payload oracle (drift guard: the schema and this table
            // must agree or someone renamed a section-8 field unilaterally).
            var payloads = new Dictionary&lt;string, Dictionary&lt;string, string&gt;&gt;
            {
                { EventIds.LevelStart, LevelFields() },
                { EventIds.LevelComplete, LevelFields() },
                { EventIds.LevelFail, LevelFields() },
                { EventIds.LineSnap, new Dictionary&lt;string, string&gt; { { "level_id", "L01" }, { "tension_band", "Sn
                { EventIds.EraserEat, new Dictionary&lt;string, string&gt; { { "level_id", "L01" }, { "anchor_id", "Top@
                { EventIds.Ad, new Dictionary&lt;string, string&gt; { { "category", "rescue" } } },
                { EventIds.Share, new Dictionary&lt;string, string&gt; { { "level_id", "L01" }, { "card_type", "onestrok
            };
            foreach (var kv in payloads)
            {
                telemetry.Track(kv.Key, kv.Value);
                Assert.AreEqual(0, telemetry.LastWarnings.Count, kv.Key + " must validate with its section-8 payload");
            }
        }
        internal static TelemetryService Service(CaptureSink sink)
        {
            var telemetry = new TelemetryService();
            if (sink != null) telemetry.AddSink(sink);
            return telemetry;
        }
        static Dictionary&lt;string, string&gt; LevelFields()
        {
            return new Dictionary&lt;string, string&gt;
            {
                { "level_id", "L01" }, { "ink_used", "2.4" }, { "attempts", "1" },
            };
        }
    }
    /// &lt;summary&gt;
```

```
/// GAP-E share-card suite (task S2-T007 slice A, G12SaveTests paradigm):
/// the generate -&gt; import -&gt; recompute round-trip end to end - the fixture
/// replays as a cleared one-stroke badge run, the claim is stamped from the
/// recompute (a code from EncodeVerified is true by construction), garbage
/// and tamper die at the decode/verify gates, the help run claims an honest
/// fail, the dry-pen law mirrors into the replay, and both card specs carry
/// the code in their Query with copy from the i18n table.&lt;/summary&gt;
public sealed class G12ShareCardTests
{
    static G12GameCatalog Catalog()
    {
        var catalog = G12GameCatalog.Load();
        Assert.NotNull(catalog, "cfg_g12_levels.json loadable from Resources");
        return catalog;
    }
    static LevelData LevelL01(G12GameCatalog catalog)
    {
        var level = catalog.LevelById("L01");
        Assert.NotNull(level, "fixture level L01 in the catalog");
        return level;
    }
    [Test]
    public void OneStroke_Fixture_Replays_Cleared_With_Badge_And_One_Star()
    {
        var verdict = G12ShareReplay.Replay(LevelL01(Catalog()), new G12ShareRun
        {
            LevelId = "L01",
            Strokes = G12ShareSchema.OneStrokeFixture(),
        });
        Assert.NotNull(verdict);
        Assert.IsTrue(verdict.Cleared, "the single lane-covering stroke clears all three waves");
        Assert.IsTrue(verdict.OneStroke, "exactly one player draw");
        Assert.AreEqual(1, verdict.Stars, "2.10 ink vs thresholds [2.25, 1.89, 1.53]");
        Assert.AreEqual(1, verdict.PlayerDraws);
    }
    [Test]
    public void EncodeVerified_Stamps_The_Claim_From_The_Recompute()
    {
        var catalog = Catalog();
        var level = LevelL01(catalog);
        string code = G12ShareCode.EncodeVerified(level, G12ShareSchema.OneStrokeFixture());
        Assert.AreEqual("LR1|L01|c1s1b1i2.10|p,2.20,2.20,4.30,B", code,
            "quantized fixture encodes byte-stable (invariant culture)");
        var run = G12ShareCode.Decode(code);
        var verdict = G12ShareReplay.Replay(level, run);
        Assert.IsTrue(run.ClaimCleared &amp;&amp; verdict.Cleared);
        Assert.IsTrue(run.ClaimOneStroke &amp;&amp; verdict.OneStroke);
        Assert.AreEqual(verdict.Stars, run.ClaimStars);
        Assert.LessOrEqual(Mathf.Abs(run.ClaimInkUsed - verdict.InkUsed), G12ShareReplay.InkTolerance);
    }
```

```
    [Test]
    public void RoundTrip_Verifies_True()
    {
        var catalog = Catalog();
        var level = LevelL01(catalog);
        string code = G12ShareCode.EncodeVerified(level, G12ShareSchema.OneStrokeFixture());
        Assert.IsTrue(G12ShareReplay.Verifies(catalog, code),
            "generate -&gt; import -&gt; recompute equals the claimed verdict");
        // byte-stable re-encode of the imported run
        Assert.AreEqual(code, G12ShareCode.EncodeRaw(G12ShareCode.Decode(code)));
    }
    [Test]
    public void Garbage_And_Truncation_Decode_To_Null()
    {
        Assert.IsNull(G12ShareCode.Decode(null));
        Assert.IsNull(G12ShareCode.Decode(""));
        Assert.IsNull(G12ShareCode.Decode("garbage"));
        Assert.IsNull(G12ShareCode.Decode("LR2|L01|c1s1b1i2.10|p,2.20,2.20,4.30,B"),
            "unknown future version = honest reject");
        Assert.IsNull(G12ShareCode.Decode("LR1|L01|c1s1b1i2.10"),
            "a truncated code never half-parses");
        Assert.IsNull(G12ShareCode.Decode("LR1|L01|c1s1b1i2.10|p,2.20,2.20,4.30,X"),
            "unknown layer char rejected");
    }
    [Test]
    public void Tampered_Claims_Fail_The_Recompute()
    {
        var catalog = Catalog();
        var level = LevelL01(catalog);
        string code = G12ShareCode.EncodeVerified(level, G12ShareSchema.OneStrokeFixture());
        var starLiar = G12ShareCode.Decode(code);
        starLiar.ClaimStars = 3;   // three stars for a one-star run
        Assert.IsFalse(G12ShareReplay.Verifies(catalog, G12ShareCode.EncodeRaw(starLiar)));
        var badgeLiar = G12ShareCode.Decode(code);
        badgeLiar.ClaimOneStroke = false;
        Assert.IsFalse(G12ShareReplay.Verifies(catalog, G12ShareCode.EncodeRaw(badgeLiar)));
        var levelLiar = G12ShareCode.Decode(code);
        levelLiar.LevelId = "L99_tampered";
        Assert.IsFalse(G12ShareReplay.Verifies(catalog, G12ShareCode.EncodeRaw(levelLiar)));
    }
    [Test]
    public void Help_Run_Claims_An_Honest_Fail()
    {
        var catalog = Catalog();
        var level = LevelL01(catalog);
        string code = G12ShareCode.EncodeVerified(level, G12ShareSchema.HelpMeFixture());
        Assert.IsTrue(G12ShareReplay.Verifies(catalog, code),
            "a true fail claim verifies - the ask premise is recomputable");
        var run = G12ShareCode.Decode(code);
        Assert.AreEqual("LR1|L01|c0s0b0i0.00|", code, "zero-stroke run encodes the empty segment");
```

```
    Assert.IsFalse(run.ClaimCleared);
    Assert.AreEqual(0, run.ClaimStars);
    Assert.AreEqual(0, run.Strokes.Length);
}
[Test]
public void Ink_Gate_Dry_Pen_Draws_Nothing_In_Replay()
{
    var level = LevelL01(Catalog());
    var verdict = G12ShareReplay.Replay(level, new G12ShareRun
    {
        LevelId = "L01",
        Strokes = new[]
        {
            new G12ShareStroke { TimeSec = 2.20f, X0 = 2.20f, X1 = 4.20f, TopLayer = false },
            new G12ShareStroke { TimeSec = 2.40f, X0 = 1.00f, X1 = 3.00f, TopLayer = false },
        },
    });
    Assert.NotNull(verdict);
    Assert.AreEqual(1, verdict.PlayerDraws, "the second 2.0 stroke cannot fit the 0.4 ink left");
    Assert.IsTrue(verdict.OneStroke);
    Assert.LessOrEqual(Mathf.Abs(verdict.InkUsed - 2.0f), G12ShareReplay.InkTolerance);
}
[Test]
public void Badge_Card_Spec_Carries_Code_And_Copy()
{
    var catalog = Catalog();
    var level = LevelL01(catalog);
    string code = G12ShareCode.EncodeVerified(level, G12ShareSchema.OneStrokeFixture());
    var card = G12ShareCards.OneStrokeBadgeCard(level, G12ShareCode.Decode(code));
    Assert.NotNull(card);
    Assert.AreEqual(code, card.Query, "the Query field rides the run code");
    Assert.AreEqual(G12ShareCards.TemplateOneStroke, card.TemplateKey);
    Assert.AreEqual(4, card.Lines.Length);
    Assert.AreEqual(G12Text.Get("ui.share.onestroke.title"), card.Title);
    Assert.AreEqual(G12Text.Get(level.nameKey), card.Lines[0]);
    Assert.AreEqual(G12Text.Get("ui.app.slogan"), card.Footer);
}
[Test]
public void Help_Card_Spec_Carries_Code_And_Copy()
{
    var catalog = Catalog();
    var level = LevelL01(catalog);
    string code = G12ShareCode.EncodeVerified(level, G12ShareSchema.HelpMeFixture());
    var card = G12ShareCards.HelpMeCard(level, G12ShareCode.Decode(code));
    Assert.NotNull(card);
    Assert.AreEqual(code, card.Query);
    Assert.AreEqual(G12ShareCards.TemplateHelpMe, card.TemplateKey);
    Assert.AreEqual(G12Text.Get("ui.share.helpme.title"), card.Title);
    Assert.AreEqual(2, card.Lines.Length);
    Assert.AreEqual(G12Text.Get(level.nameKey), card.Lines[0]);
```

```
            Assert.AreEqual(G12Text.Get("ui.share.helpme.sub"), card.Lines[1]);
        }
        [Test]
        public void Badge_Card_Refuses_A_Run_Without_The_Badge_Claim()
        {
            var catalog = Catalog();
            var level = LevelL01(catalog);
            string code = G12ShareCode.EncodeVerified(level, G12ShareSchema.HelpMeFixture());
            var run = G12ShareCode.Decode(code);
            Assert.IsNull(G12ShareCards.OneStrokeBadgeCard(level, run),
                "the flex seat never ships a run the badge law did not award");
        }
        [Test]
        public void Dock_Lands_Both_Card_Types()
        {
            var sink = new CaptureSink();
            var events = new G12TelemetryEvents(G12TelemetryTests.Service(sink));
            events.NotifyShare("L01", G12ShareCards.CardTypeOneStroke);
            events.NotifyShare("L02", G12ShareCards.CardTypeHelpMe);
            Assert.AreEqual(2, sink.Ids.Count);
            Assert.AreEqual(EventIds.Share, sink.Ids[0]);
            Assert.AreEqual(EventIds.Share, sink.Ids[1]);
            Assert.AreEqual("onestroke", sink.Fields[0]["card_type"]);
            Assert.AreEqual("helpme", sink.Fields[1]["card_type"]);
        }
        [Test]
        public void Code_Is_Printable_Ascii_Only()
        {
            var catalog = Catalog();
            var level = LevelL01(catalog);
            string code = G12ShareCode.EncodeVerified(level, G12ShareSchema.OneStrokeFixture());
            foreach (char c in code)
            {
                Assert.IsTrue(c &gt; ' ' &amp;&amp; c &lt;= (char)0x7E,
                    "payload char must stay printable ASCII: " + (int)c);
            }
        }
    }
}
// G12VerdictOracleTests.cs
using System;
using System.Collections.Generic;
using System.Text;
using NUnit.Framework;
using Sy.G12;
namespace Sy.G12.Tests
{
    /// &lt;summary&gt;
    /// S2-T009 verdict-law oracle (07 S2 face: "verdict law - Oracle injects
    /// rng modes"). The oracle re-derives the verdict law from PUBLIC faces
```

```
/// only (event log + counters + line state), so it stays an honest
/// independent check: any drift between law and log breaks here first.
/// The rng seam on LineRescueSim lets tests sweep the same law under
/// injected chaos lane/weight patterns (deterministic by default).
/// &lt;/summary&gt;
public static class G12VerdictOracle
{
    public static List&lt;string&gt; Problems(LineRescueSim sim)
    {
        var p = new List&lt;string&gt;();
        int dead = 0, eaten = 0;
        for (int i = 0; i &lt; sim.Lines.Count; i++)
        {
            var segs = sim.Lines[i].Segs;
            for (int s = 0; s &lt; segs.Count; s++)
            {
                if (segs[s].Dead) dead++;
                if (segs[s].Eaten) eaten++;
            }
        }
        if (sim.Saves != sim.CountEvents("rescue_success")) p.Add("saves!=rescue_success events");
        // AcceptRevive/AcceptRetry reset Fails to zero while the event log
        // stays append-only; each exit happened at Fails==MaxFails (Failed
        // phase stops fail production), so events = Fails +
        // (ReviveCount + FreeRetryCount) * MaxFails (v1.1 C1296: the free
        // retry exits absorb the same staircase a revive absorbs).
        if (sim.Fails + (sim.ReviveCount + sim.FreeRetryCount) * LineRescueSim.MaxFails != sim.CountEvents("rescue_f
        if (sim.DrawCount != sim.CountEvents("line_ready")) p.Add("drawCount!=line_ready events");
        if (sim.HintCount != sim.CountEvents("hint_line_shown")) p.Add("hintCount!=hint_line_shown events");
        if (sim.CountEvents("line_snap") != dead) p.Add("line_snap events!=dead segs");
        if (sim.CountEvents("eraser_eat") != eaten) p.Add("eraser_eat events!=eaten segs");
        if (sim.CurrentPhase == LineRescueSim.Phase.Failed)
        {
            if (sim.Fails &lt; LineRescueSim.MaxFails) p.Add("failed phase with fails&lt;MaxFails");
            if (sim.CountEvents("revive_offer") &lt; 1) p.Add("failed phase without revive_offer");
        }
        if (sim.ReviveCount &gt; sim.CountEvents("revive_offer")) p.Add("revives&gt;offers");
        return p;
    }
    public static void AssertLaw(LineRescueSim sim)
    {
        List&lt;string&gt; probs = Problems(sim);
        Assert.IsEmpty(probs, "verdict-law oracle: " + string.Join("; ", probs.ToArray()));
    }
}
/// &lt;summary&gt;Chaos rng policy for oracle sweeps: reversed lane cycle and a
/// weight ladder, both fully scripted so the test can mirror the pattern.
/// Max weight 2.0 vs hint cap 2.5 keeps every resting load in Hold.&lt;/summary&gt;
public class G12ChaosRng : LineRescueSim.IRngSource
{
```

```
    public int Calls;
    public float NextLane(int waveIndex)
    {
        Calls++;
        int m = waveIndex % 3;
        return m == 0 ? 4.0f : (m == 1 ? 2.5f : 1.0f);
    }
    public float NextWeight(int waveIndex)
    {
        return 0.6f + (waveIndex % 5) * 0.35f;
    }
}
[TestFixture]
public class G12VerdictOracleTests
{
    static void Advance(LineRescueSim sim, float seconds)
    {
        int steps = (int)Math.Round(seconds / 0.05f);
        for (int i = 0; i &lt; steps; i++) sim.AdvanceClock(0.05f);
    }
    static string EventKinds(LineRescueSim sim)
    {
        var sb = new StringBuilder();
        for (int i = 0; i &lt; sim.Events.Count; i++)
        {
            if (i &gt; 0) sb.Append(',');
            sb.Append(sim.Events[i].Kind);
        }
        return sb.ToString();
    }
    [Test]
    public void DeterministicDefault_OracleHoldsAndReplayIsStable()
    {
        Assert.AreEqual(LineRescueSim.RngMode.Deterministic, new LineRescueSim().Rng, "fresh sim must default to Det
        var a = new LineRescueSim();
        a.Open();
        Advance(a, 7f);
        Assert.AreEqual(LineRescueSim.Phase.Failed, a.CurrentPhase, "no-line run must fail out at MaxFails");
        Assert.AreEqual(LineRescueSim.MaxFails, a.Fails, "three drops = three fails");
        G12VerdictOracle.AssertLaw(a);
        var b = new LineRescueSim();
        b.Open();
        Advance(b, 7f);
        G12VerdictOracle.AssertLaw(b);
        Assert.AreEqual(EventKinds(a), EventKinds(b), "default mode must replay identically (E4 anchor law)");
    }
    [Test]
    public void InjectedChaosSweep_OracleHoldsAndSourceWasConsulted()
    {
        var sim = new LineRescueSim();
```

```
            var rng = new G12ChaosRng();
            Assert.IsTrue(sim.InjectRng(rng), "non-null policy must be accepted");
            Assert.AreEqual(LineRescueSim.RngMode.Injected, sim.Rng, "mode must flip to Injected");
            sim.Open();
            Advance(sim, 7f);
            Assert.GreaterOrEqual(rng.Calls, 3, "injected source must be consulted per spawned wave");
            Assert.AreEqual(4.0f, sim.Hazards[0].X, 0.0001f, "wave 0 lane must come from the injected policy");
            G12VerdictOracle.AssertLaw(sim);
            Assert.IsTrue(sim.ReviveOfferShown, "fail-out must show the revive seat");
            sim.AcceptRevive();
            Assert.AreEqual(1, sim.ReviveCount, "revive seat accepted");
            for (int i = 0; i &lt; 40; i++)
            {
                sim.AdvanceClock(0.05f);
                if (sim.CurrentPhase == LineRescueSim.Phase.Playing) sim.RequestHint();
            }
            G12VerdictOracle.AssertLaw(sim);
        }
        [Test]
        public void ClearInjectedRng_RestoresDeterministicLanePattern()
        {
            var sim = new LineRescueSim();
            Assert.IsFalse(sim.InjectRng(null), "null is not a policy and must be refused");
            Assert.AreEqual(LineRescueSim.RngMode.Deterministic, sim.Rng, "refused injection must not flip the mode");
            Assert.IsTrue(sim.InjectRng(new G12ChaosRng()));
            Advance(sim, 1.5f);
            Assert.AreEqual(4.0f, sim.Hazards[0].X, 0.0001f, "injected lane pattern visible on wave 0");
            sim.ClearInjectedRng();
            Assert.AreEqual(LineRescueSim.RngMode.Deterministic, sim.Rng, "clear must restore Deterministic");
            Advance(sim, 2f);
            Assert.AreEqual(2.5f, sim.Hazards[1].X, 0.0001f, "wave 1 must return to the legacy lane cycle");
            G12VerdictOracle.AssertLaw(sim);
        }
        [Test]
        public void Oracle_CatchesTamperedEventLog()
        {
            var sim = new LineRescueSim();
            sim.Open();
            Advance(sim, 7f);
            G12VerdictOracle.AssertLaw(sim);
            var evs = (List&lt;LineRescueSim.Ev&gt;)sim.Events;
            evs.Add(new LineRescueSim.Ev { Kind = "rescue_fail", SimSeconds = sim.Clock });
            List&lt;string&gt; probs = G12VerdictOracle.Problems(sim);
            Assert.IsNotEmpty(probs, "oracle must catch a forged rescue_fail event");
            bool sawFailsLaw = false;
            for (int i = 0; i &lt; probs.Count; i++) if (probs[i].Contains("fails")) sawFailsLaw = true;
            Assert.IsTrue(sawFailsLaw, "the fails!=rescue_fail law must be among the reported problems");
        }
    }
}
```